

Pipelining

"Nunca perca tempo", Provérbio americano

*** O resumo a seguir pode apresentar erros; caso ache algo estranho ou que você ache que está errado, não hesite em avisar ao monitor pelo número (37)99995-0154 ou pelo e-mail rafaelpereira1000@gmail.com; se não for um erro, é uma boa oportunidade para tirar dúvidas! O resumo foi feito com base no livro da disciplina e não foi colocado tudo que está nos slides de aula, por isso sempre confira nos slides (e no livro) todo o conteúdo em paralelo caso você vá usar o resumo! Bons estudos a todos! ***

Conteúdo

1	Visão geral de pipelining	2
1.1	Hazards do Pipeline	4
2	Caminho de dados e controle usando pipeline	9
2.1	Controle de um pipeline	15
3	Hazards de dados: forwarding vs stalls	18
3.1	Hazards de dados e stalls	24
4	Hazards de controle	26
4.1	Reduzindo o atraso dos desvios	27
4.2	Previsão dinâmica de desvios	30
4.3	Escalonamento com delay slot do desvio	32
5	Exceções	33
5.1	Como tratar exceções no MIPS	34
5.2	Exceções em uma implementação em pipeline	34
6	Estendendo o pipeline para lidar com operações de Ponto Flutuante	38
6.1	Hazards e adiamentos nos pipelines de latência maior	40
6.2	Hazards de dados: RAW, WAW e WAR	41

1 Visão geral de pipelining

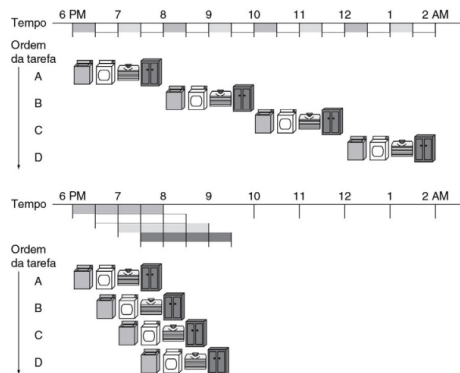


Figura 1: A analogia da lavagem de roupas para pipelining: Ana, Beto, Catarina e Davi possuem roupas sujas para serem lavadas, secadas, passadas e guardadas: a lavagem sequencial levaria 8 horas para quatro trouxas de roupas, enquanto a lavagem com pipeline levaria apenas 3,5 horas

⇒ O paradoxo da técnica de pipelining é que o tempo desde a colocação de uma única trouxa de roupa suja na lavadora até que ela esteja seca, e seja passada e guardada não é mais curto (o processo) para tal técnica

⇒ O motivo pelo qual a técnica de pipelining é mais rápida para muitas trouxas é que tudo está trabalhando em paralelo, de modo que mais trouxas são terminadas por hora → a técnica de pipelining melhora a vazão do sistema de lavanderia sem melhorar o tempo para concluir uma única trouxa

⇒ Nos processadores, a técnica de pipelining permite que várias instruções sejam sobrepostas na execução → melhora a vazão do sistema

⇒ O tempo individual na execução de uma instrução não muda, mas o tempo de execução de várias instruções (throughput) é potencialmente menor

⇒ O efeito do início e o fim (quando o pipeline ainda não está cheio) afeta o desempenho quando o número de tarefas não é grande em comparação com a quantidade de estágios do pipeline

⇒ A técnica de Pipling explora o paralelismo em nível de instrução (ILP - Instruction Level Parallelism) e é a principal técnica de uma geração de microprocessadores para executarem programas mais rapidamente

■ Etapas para instruções do MIPS:

1. Buscar a instrução da memória
2. Ler registradores enquanto a instrução é decodificada (o formato das instruções do MIPS permite que a leitura e a decodificação ocorram simultaneamente)
3. Executar a operação ou calcular um endereço
4. Acessar um operando na memória de dados
5. Escrever o resultado em um registrador

■ Exemplo:

Compare o tempo médio entre as instruções de uma implementação em ciclo único, em que todas as instruções levam um ciclo de clock, com uma implementação com pipeline, dados os tempos de operação para as principais unidades funcionais abaixo:

Classe de instrução	Busca de instrução	Leitura do registrador	Operação ALU	Acesso de dados	Escrita do registrador	Tempo Total
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR, slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

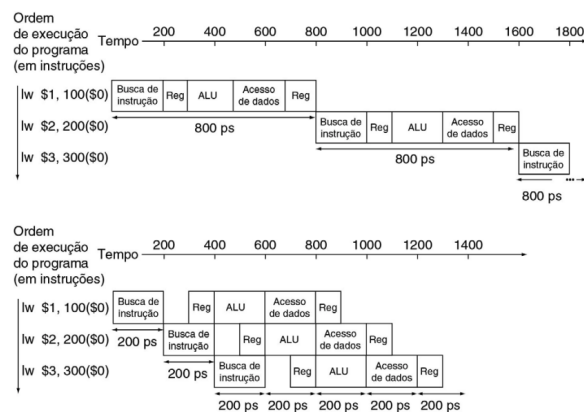


Figura 2: Em cima, execução em ciclo único, sem pipeline, vs execução com pipeline (embaixo)

No modelo de ciclo único, cada instrução leva exatamente um ciclo de clock, de modo que o ciclo precisa ser esticado para acomodar a instrução mais lenta

Desse modo, o tempo entre a 1ª e a 4ª instruções no projeto sem pipeline é 3 x 800ps, ou 2.400ps

Todos os estágios do pipeline utilizam um único ciclo de clock, de modo que ele precisa ser grande o suficiente para acomodar a operação mais lenta

Assim como o projeto de ciclo único de clock precisa levar o tempo do ciclo de clock no pior caso, de 800ps, embora algumas instruções possam ser tão rápidas quanto 500ps, o ciclo de clock da execução com pipeline precisa ter o ciclo de clock no pior caso de 200ps, embora alguns estágios levem apenas 100ps

Mesmo assim, uso de pipeline ainda oferece uma melhoria de desempenho de 4 vezes: o tempo entre a 1ª e a 4ª instruções é de 3 x 200ps, ou 600ps

■ Ganho de velocidade com a técnica de pipelining:

$$Tempo_{ideal\ entre\ instruções\ pipeline} = \frac{Tempo_{ideal\ entre\ instruções\ uniciclo}}{N^{\circ}\ de\ estágios\ do\ pipeline}$$

⇒ Idealmente, um pipeline de 5 estágios levaria a um ganho de 5 vezes, porém:

Os estágios não são balanceados (o tempo de cada estágio não é igual à divisão do tempo total do uniciclo pelo número de estágios) → no exemplo anterior, o acesso ao Banco de Registradores leva metade de um ciclo, fator que permite a leitura de registradores na 2ª metade do ciclo e a escrita na 1ª

Há o overhead de encher o pipeline

⇒ Assim, o pipeline melhora a vazão do sistema em vez de diminuir o tempo de execução de uma instrução individual → a vazão de instruções é a medida importante, pois os programas reais executam bilhões de instruções

■ Continuação do exemplo:

Além do mais, até mesmo nossa afirmação de uma melhoria de 4 vezes para nosso exemplo não está refletida no tempo de execução total para as 3 instruções: são 1.400ps vs 2.400ps

Naturalmente, isso acontece porque o número de instruções não é grande. O que aconteceria se aumentássemos o número de instruções?

Poderíamos estender os valores anteriores para 1.000.003 instruções. Acrescentaríamos 1.000.000 instruções no exemplo com pipeline; cada instrução acrescenta 200ps ao tempo de execução total.

O tempo de execução total seria:

$$1.000.000 \times 200\text{ps} + 1.400\text{ps}, \text{ ou } 200.001.400\text{ps}$$

No exemplo sem pipeline, acrescentaríamos 1.000.000 instruções, cada uma exigindo 800ps, de modo que o tempo de execução total seria:

$$1.000.000 \times 800\text{ps} + 2.400\text{ps}, \text{ ou } 800.002.400\text{ps}$$

Sob essas condições ideais, a razão entre os tempos de execução total para os programas reais nos processadores sem pipeline e com pipeline é próxima da razão de tempos entre as instruções:

$$\frac{800.002.400 \text{ ps}}{200.001.400 \text{ ps}} \approx \frac{800 \text{ ps}}{200 \text{ ps}} \approx 4.00$$

■ Projetando um conjunto de instruções para o pipeline:

1. Todas as instruções do MIPS têm o mesmo tamanho → torna muito mais fácil buscar instruções no 1º estágio do pipeline e decodificá-las no 2º estágio
2. O MIPS tem apenas alguns formatos de instruções → O 2º estágio pode começar a ler o Banco de Registradores ao mesmo tempo em que o hardware está determinando que tipo de instrução foi lida
3. Os operandos em memória aparecem somente em lw ou sw no MIPS → podemos usar o estágio de execução (3º) para calcular o endereço de memória e depois acessar a memória no estágio seguinte
4. Os operandos precisam estar alinhados na memória → não precisamos nos preocupar com uma única instrução de transferência de dados exigindo dois acessos à memória de dados

1.1 Hazards do Pipeline

⇒ Situações em que a instrução não pode ser executada no ciclo de clock seguinte por causa de conflitos → conflitos causam paradas no pipeline (**stalls/bolhas**) a fim de resolvê-los

⇒ Todas as instruções buscadas depois do stall são adiadas → nenhuma instrução nova é apanhada durante o stall

⇒ As instruções buscadas antes do stall devem continuar!

Hazards estruturais: ocorrem quando o hardware não pode admitir a combinação de instruções que queremos executar no mesmo ciclo de clock

⇒ Ocorreriam caso fosse usada somente uma memória para Dados e Instruções

⇒ O Banco de Registradores não gera hazard estrutural em nossa implementação já que nele a escrita ocorre na 1ª metade do ciclo de clock e a leitura na 2ª metade

Hazards de dados: ocorrem quando uma instrução depende do resultado de uma instrução anterior que ainda está no pipeline

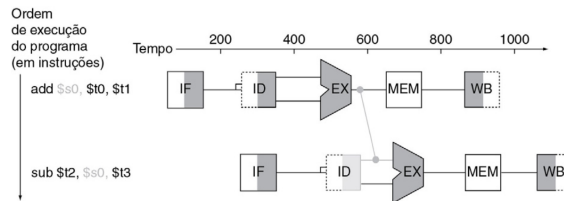
Ex: add \$s0, \$t0, \$t1
 sub \$t2, \$s0, \$t3

⇕

A instrução add só escreve seu resultado no 5º estágio, significando que precisaríamos de 3 bolhas

⇕

Solução: Assim que a ALU cria a soma para o add, podemos fornecê-la como entrada para a subtração



⇒ **Forwarding / Bypassing:** acréscimo de hardware extra para ter o item que falta antes do previsto, diretamente de recursos internos, como no exemplo acima

⇒ Os caminhos do forwarding só são válidos se estágio de destino estiver mais adiante no tempo que o estágio de origem

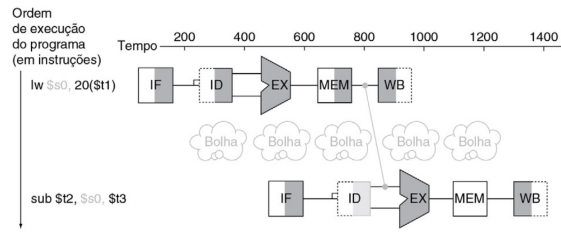
Ex: lw \$s0, 20(\$t1)
 sub \$t2, \$s0, \$t3

⇕

Os dados em \$s0 só estarão disponíveis depois do 4º estágio

⇕

deve ocorrer um "pipeline stall/bolha", mesmo com o uso de forwarding



Podemos tratar usando a detecção de hardware e stalls ou software que reordena o código para evitar esses casos de lw, como no exemplo abaixo

⇒ Exemplo:

Considere o seguinte segmento de código em C:

a = b + e;
c = b + f;

Temos o código MIPS gerado para esse segmento, supondo que todas as variáveis estejam na memória e sejam endereçáveis como offsets a partir de \$t0

As duas instruções add possuem um hazard, devido à respectiva dependência da instrução lw imediatamente anterior

Observe que o bypassing elimina vários outros hazards em potencial, incluindo a dependência do primeiro add no primeiro lw e quaisquer hazards para instruções store. Subir a terceira instrução lw elimina os dois hazards

lw	\$t1, 0(\$t0)	lw	\$t1, 0(\$t0)
lw	<u>\$t2</u> , 4(\$t0)	lw	\$t2, 4(\$t0)
add	\$t3, \$t1, <u>\$t2</u>	lw	\$t4, 8(\$t0)
sw	\$t3, 12(\$t0)	add	\$t3, \$t1, \$t2
lw	<u>\$t4</u> , 8(\$t0)	sw	\$t3, 12(\$t0)
add	\$t5, \$t1, <u>\$t4</u>	add	\$t5, \$t1, \$t4
sw	\$t5, 16(\$t0)	sw	\$t5, 16(\$t0)

(a) Antes

(b) Depois

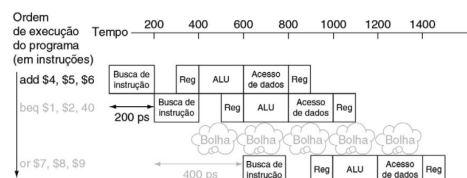
Figura 3: Reordenação de código para evitar stalls

Cada instrução MIPS escreve no máximo um resultado e faz isso no último estágio do pipeline

Hazards de controle: vêm da necessidade de tomar uma decisão com base nos resultados de uma instrução enquanto outras estão sendo executadas

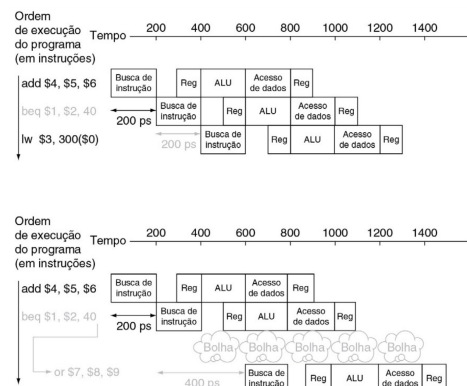
⇒ Ocorrem em instruções de desvio, ou seja, que alteram o PC: beq, bne, jump

⇒ Uma solução é colocarmos uma bolha imediatamente após buscarmos um desvio (esperando o pipeline determinar o seu resultado para saber qual o endereço da próxima instrução)



⇒ Outra solução é usar a previsão:

⇒ Podemos prever que os desvios não serão tomados; quando realmente não forem tomados, o pipeline segue normalmente e quando forem tomados, o pipeline sofre um stall:



⇒ Ou podemos prever que os desvios serão tomados, o que seria eficiente em loops, já que voltamos para um endereço anterior várias vezes

⇒ Ou usamos a previsão dinâmica que mantém um histórico para prever o futuro → quando a escolha estiver errada, o controle do pipeline deve garantir que as instruções de desvio errado não tenham efeito (reiniciando o pipeline a partir do endereço certo) → detalhes de implementação na seção 4.2

⇒ Ou podemos usar um delayed Branch (decisão adiada) no qual o software do MIPS colocará uma instrução imediatamente após a instrução de Branch, que não é afetada pelo desvio, e um desvio tomado muda o endereço da instrução que vem após essa instrução segura → o delayed Branch sempre executa a instrução sequencial (que deve ser segura) com o desvio sendo ou não tomado após esta instrução → Solução adotada pelo MIPS → detalhes de implementação na seção 4.3

■ Exemplo 1:

Em um processador, leva-se 3 estágios do pipeline antes de se determinar o endereço de destino de um desvio, e mais 1 estágio para saber se o mesmo será tomado. Quais as penalidades dos desvios (condicionais e incondicionais) nas 3 abordagens vistas (esvaziar pipeline, previsto tomado e previsto não tomado)?

Branch scheme	Penalty unconditional	Penalty untaken	Penalty taken
Flush pipeline	2	3	3
Predicted taken	2	3	2
Predicted untaken	2	0	3

⇒ Pipelines têm um limite de profundidade, pois alguma latência é introduzida entre os estágios do pipeline devido aos registradores intermediários (pipes) → quando o tempo do ciclo é tão pequeno quanto os overheads, deixa de valer a pena ter mais estágios

■ Exemplo 2:

Considere um processador multiciclo, sem pipeline. Suponha que ele tenha um ciclo de clock de 1ns e use 4 ciclos para instruções de ULA e desvios, e 5 ciclos para operações de memória. As frequências dessas instruções são 40%, 20% e 40%, respectivamente. Considere que a versão pipeline acrescenta 0,2 ns de overhead. Qual o ganho com o pipeline?

Sem pipeline:

$$\text{Tempo Médio de Execução (TME)} = \text{Ciclo de clock} \times \text{CPI}$$

$$\text{TME} = 1\text{ns} \times ((40\% + 20\%) \times 4 + 40\% \times 5) = 1\text{ns} \times 4,4$$

$$\text{TME} = 4,4\text{ ns}$$

Com pipeline:

$$\text{Clock com overhead: } 1 + 0,2 = 1,2\text{ ns}$$

$$\text{TME} = 1\text{ ciclo} = 1,2\text{ ns}$$

$$\text{Ganho (Speedup)} = \text{TME}_{\text{sempipe}} / \text{TME}_{\text{compipe}}$$

$$\text{Ganho (Speedup)} = 4,4 / 1,2 = 3,7\text{ vezes}$$

■ Desempenho com Stalls

CPI ideal do pipeline é 1 → Stalls aumentam o CPI com pipeline

Considerando somente efeito de stalls no CPI (ou seja, sem overhead de ciclo e pipeline balanceado):

$$\text{CPI}_{\text{com-pipeline}} = \text{CPI}_{\text{ideal}} + \text{Ciclos de stall por instrução}$$

$$\text{CPI}_{\text{com-pipeline}} = 1 + \text{Ciclos de stall por instrução}$$

Considerando mesmo número de ciclos por instrução, que deve ser a profundidade do pipeline:

$$Ganho(Speedup)_{pipeline} = \frac{N^{\circ} \text{ de estágios do pipeline (profundidade)}}{1 + \text{ciclos de stall por instrução}}$$

(Exemplo) Considerando stalls de controle:

$$\text{Ciclos de stall por instrução} = \text{Frequência_desvio} \times \text{Penalidade}$$

$$Ganho(Speedup)_{pipeline} = \frac{N^{\circ} \text{ de estágios do pipeline (profundidade)}}{(1 + \text{Frequência_desvio} \times \text{Penalidade})}$$

■ Exemplo 3:

Assuma que, em um programa, as instruções de referências a dados constituem 40% do total. Em um processador sem conflito estrutural de memória que executa esse programa, o CPI ideal é igual a 1. Um outro processador, com conflito estrutural de memória, tem taxa de clock 1,05 vez maior que o sem conflito. Desconsiderando quaisquer outras perdas de desempenho, quantas vezes o processador sem conflito é mais rápido que o processador com conflito?

$$\begin{aligned} \text{Tempo por instrução} &= \text{CPI} \times \text{Tempo_ciclo} \\ \text{Tempo por instrução sem conflito} &= \text{Tempo_ciclo-ideal} \\ \text{Tempo com conflito} &= (1 + 0,4 \times 1) \times \text{Tempo_ciclo-ideal} / 1,05 \\ &= 1,3 \times \text{Tempo_ciclo-ideal} \end{aligned}$$

Sem conflito é 1,3 vezes mais rápido

Observação: Alternativas seriam separar as memórias de dados e instruções, ou fazer caches independentes

■ Exemplo 4:

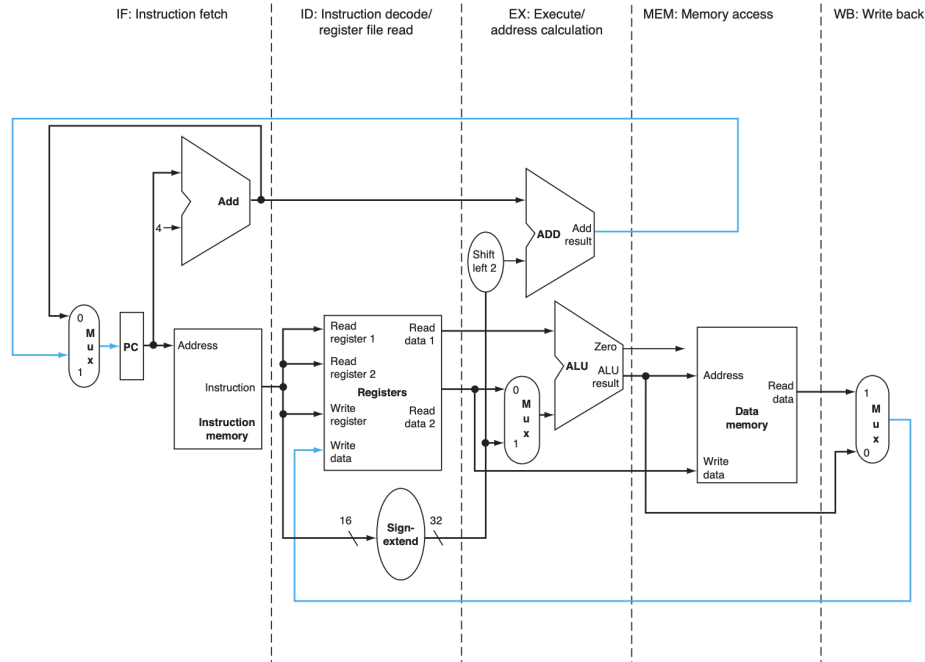
Um projetista está na dúvida entre implementar um processador com pipeline de 5 estágios, que provoca uma penalidade de 2 stalls por desvio, ou um processador com 6 estágios, mas que causa uma penalidade de 4 stalls por desvio. Qual a melhor escolha, considerando que instruções de desvio representam 20% de um programa típico?

$$\text{Ganho1} = 5 / (1 + 0,20 \times 2) = 3,57$$

$$\text{Ganho2} = 6 / (1 + 0,20 \times 4) = 3,33$$

Escolha 1!

2 Caminho de dados e controle usando pipeline



⇒ 5 estágios → até 5 instruções estarão em execução durante qualquer ciclo de clock

⇒ Estágios:

1.**IF** → Instruction Fetch (Busca de instruções)

2.**ID** → Instruction Decode (Decodificação de instruções e leitura do Banco de Registradores)

3.**EX** → Execução ou cálculo de endereço

4.**MEM** → Acesso à memória de dados

5.**WB** → Write Back (Escrita do resultado)

As instruções e os dados movem da esquerda para a direita, exceto:

■ Estágio de WB, que coloca o resultado de volta no Banco de Registradores

■ A seleção do próximo valor do PC (escolhendo entre o PC incrementado e o endereço de desvio calculado no estágio MEM)

Os dados fluindo da direita para a esquerda não afetam a instrução atual; somente as instruções seguintes no pipeline são influenciadas por esses movimentos de dados reversos

Para usar o mesmo caminho de dados na execução das instruções acrescentamos registradores para manter dados de modo que partes do caminho de dados pudessem ser compartilhadas durante a execução da instrução



Colocamos registradores sempre que existam linhas divisórias entre os estágios



Todas as instruções avançam durante cada ciclo de clock de um registrador de pipeline para o seguinte

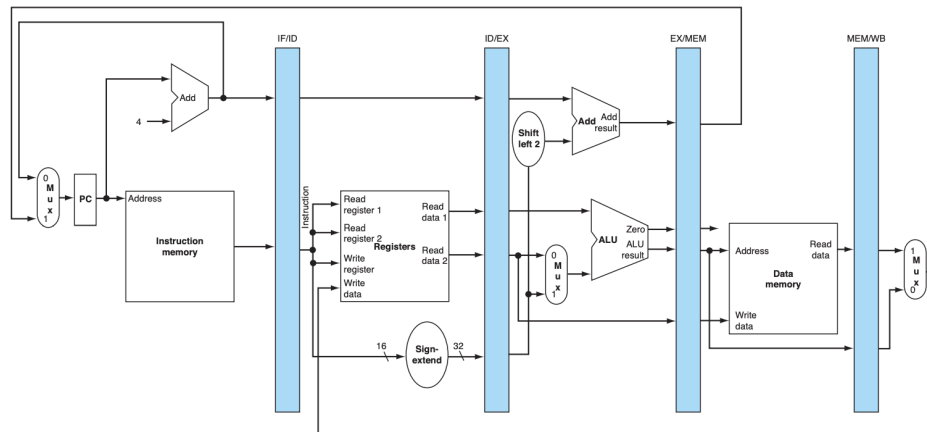


Figura 4: Os registradores precisam ser grandes o suficiente para armazenar todos os dados correspondentes às linhas que passam por eles → IF/ID precisa ter 64 bits de largura, pois precisa manter a instrução de 32 bits lida da memória e o endereço incrementado de 32 bits no PC → por enquanto, os outros três registradores de pipeline contêm 128, 97 e 64 bits, respectivamente

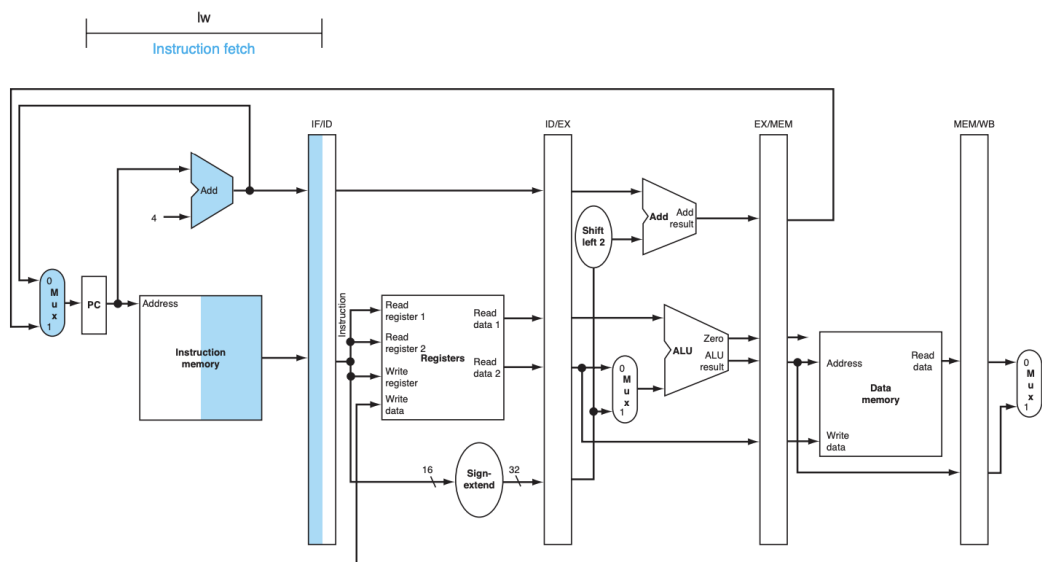
⇒ Registradores do pipeline ("pipes"):

IF/ID, ID/EX, EX/MEM, MEM/WB

⇒ O PC pode ser considerado um registrador do pipeline que "alimenta" o estágio IF → Contudo, o PC faz parte do estado arquitetônico visível; seu conteúdo precisa ser salvo quando ocorre uma exceção, enquanto o conteúdo dos registradores de pipeline pode ser descartado

⇒ Análise de uma instrução lw com pipeline a cada estágio:

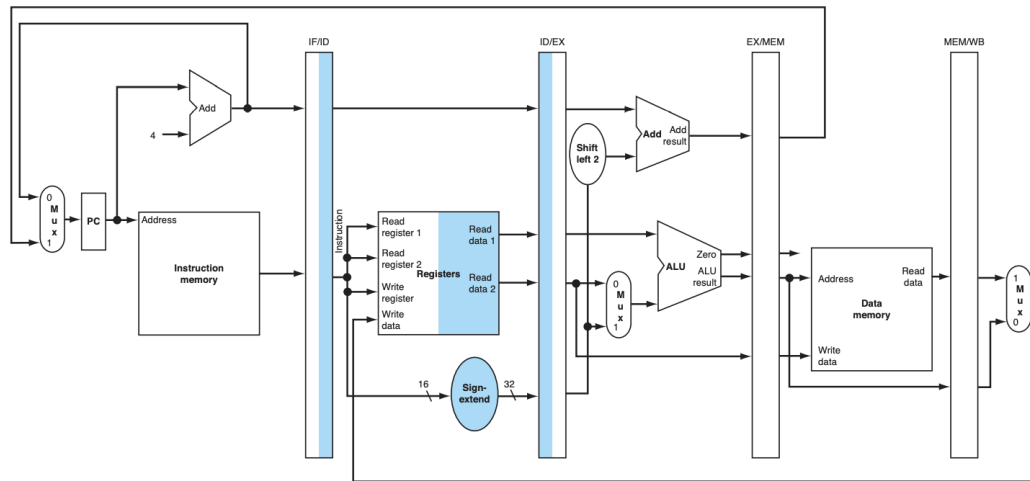
1. Busca de instruções



A instrução é lida da memória usando o endereço no PC e depois colocada no registrador de pipeline IF/ID; o endereço do PC é somado +4 e depois escrito de volta ao PC (no próximo ciclo de clock temos o endereço da próxima instrução)

PC+4 é salvo no IF/ID, pois a instrução pode ser de desvio e esse valor é necessário para calcular o desvio

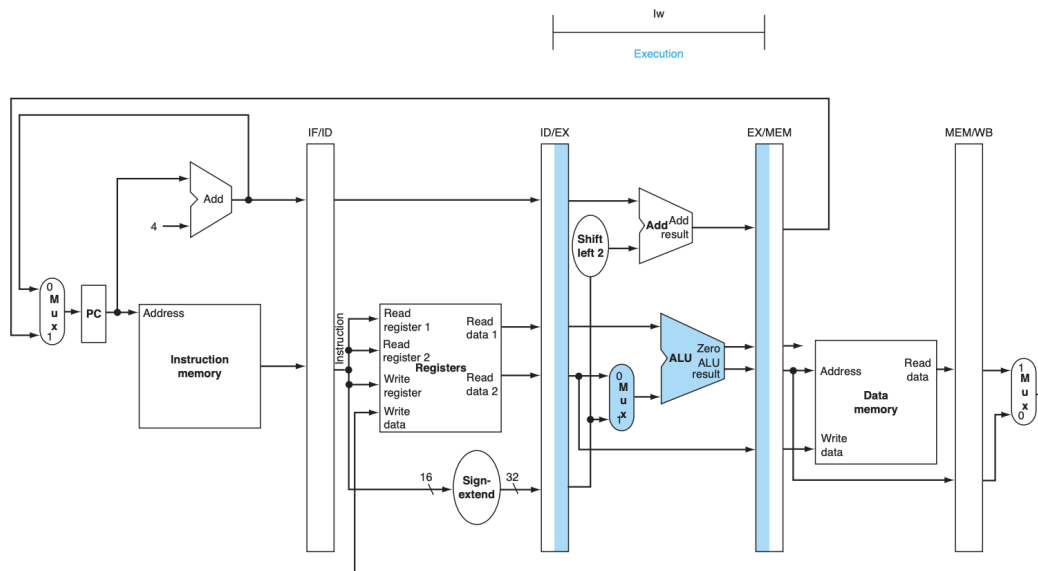
2. Decodificação de instruções e leitura do Banco de Registradores



A instrução que foi buscada no estágio anterior e que foi armazenada no pipe IF/ID fornece os números dos dois registradores para leitura no Banco de Registradores e o campo imediato de 16 bits (que tem seu sinal estendido para 32 bits)

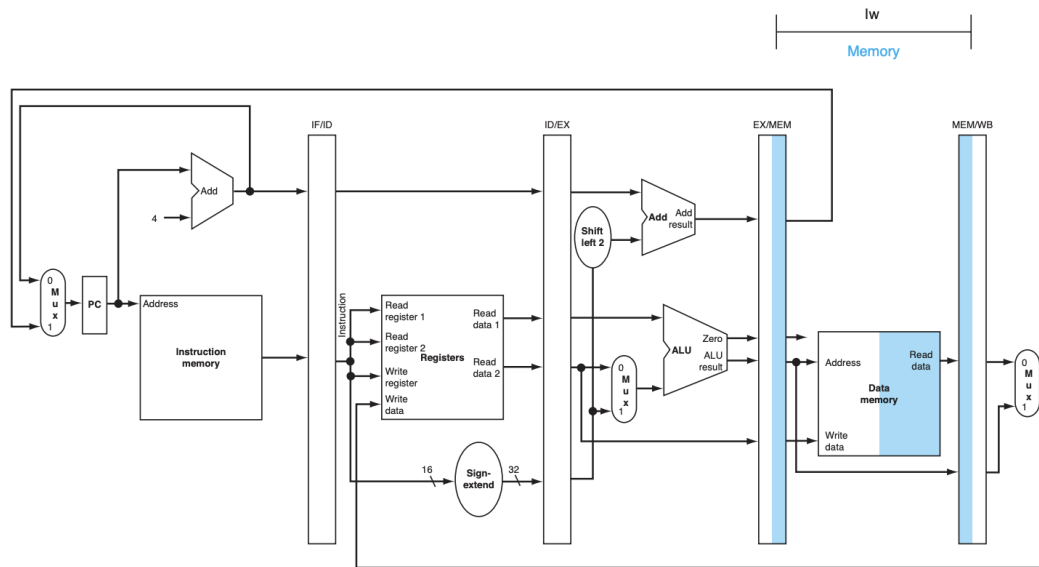
Os valores lidos do Banco de Registradores e o campo imediato estendido são armazenados no pipe ID/EX, assim como PC+4

3. Execução ou cálculo de endereço



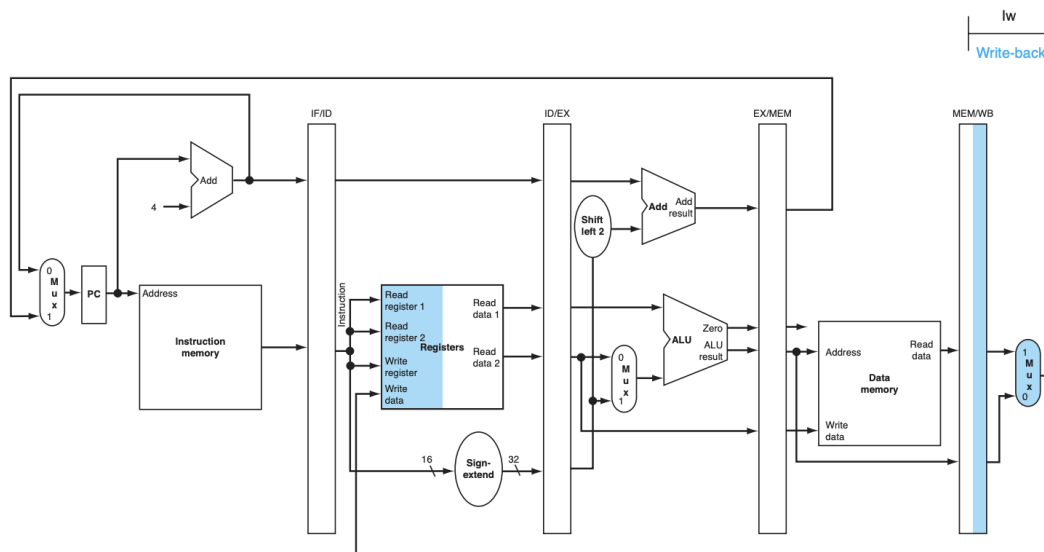
A instrução lw lê o conteúdo do registrador 1 e o imediato com o sinal estendido do registrador de pipeline ID/EX e os soma usando a ALU; essa soma é colocada no registrador de pipeline EX/MEM

4. Acesso à memória



A instrução lw lê o valor no pipe EX/MEM do endereço a ser acessado e carrega os dados lidos da memória de dados para o pipe MEM/WB

5. Escrita do resultado

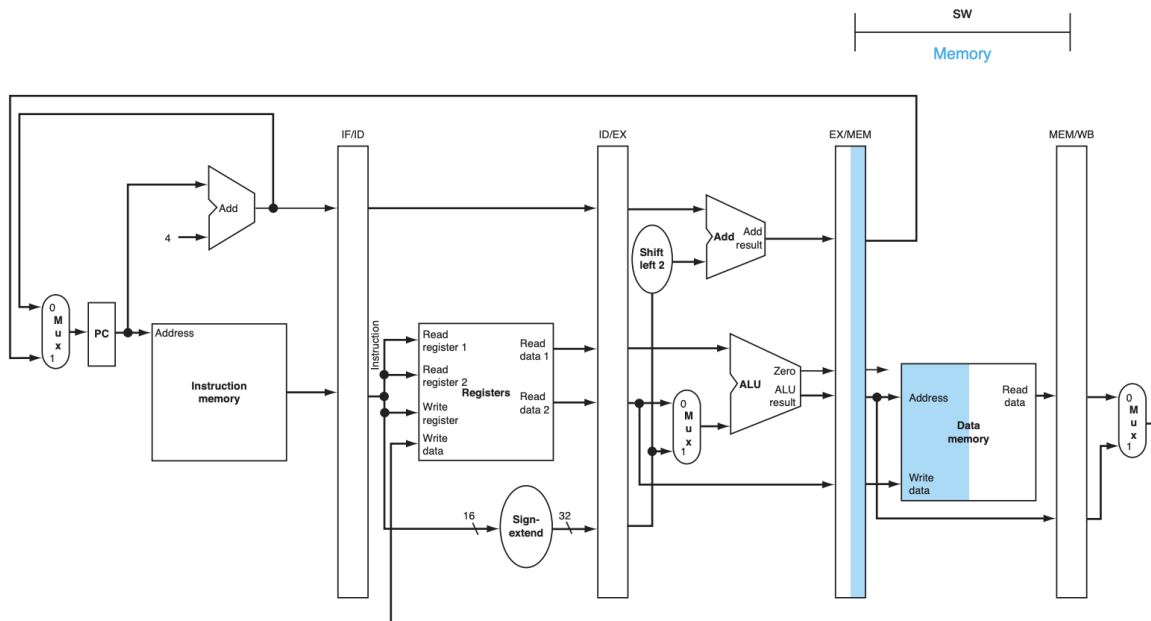


Os dados armazenados no pipe MEM/WB são lidos e escritos no Banco de Registradores

⇒ Análise de uma instrução sw com pipeline a cada estágio:

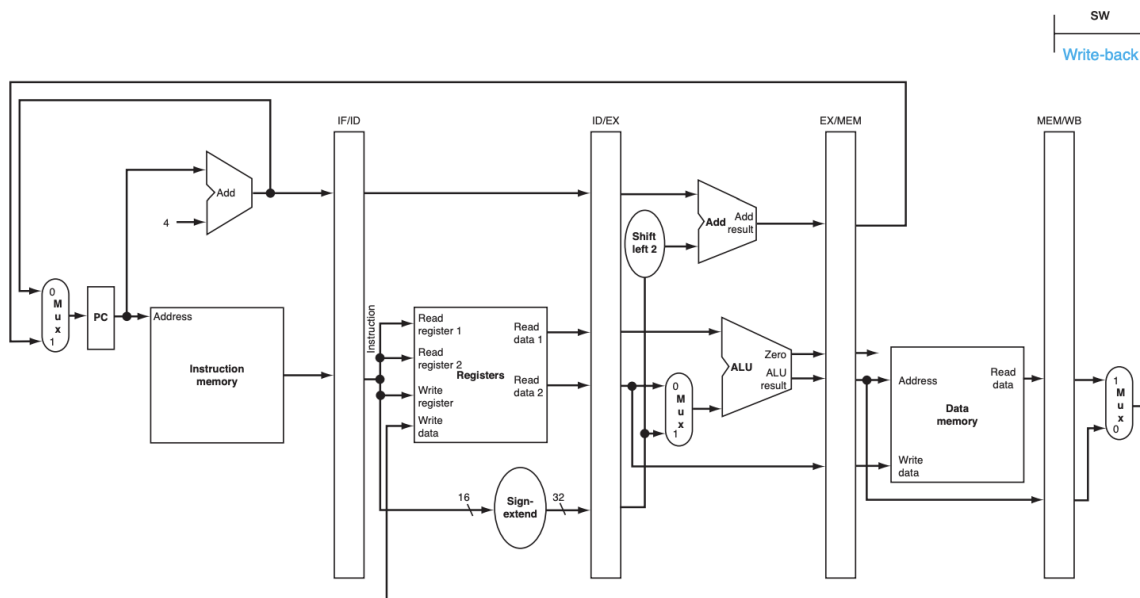
1, 2 e 3 → fazem a mesma coisa que o lw

4. Acesso à memória



O registrador contendo os dados a serem armazenados foi lido no estágio ID e esses dados foram armazenados no pipe ID/EX → A única maneira de disponibilizar os dados durante o estágio MEM é colocar esses dados no registrador de pipeline EX/MEM no estágio EX, assim como o endereço de armazenamento (calculado no estágio EX e colocado no pipe EX/MEM, da mesma forma que é feito para o lw)

5. Escrita do resultado



Nada acontece! sw passa por um estágio mesmo que não haja o que fazer, e as instruções posteriores ao sw continuam prosseguindo

⇒ Cada componente lógico do caminho de dados – como memória de instruções, portas para leitura de registradores, ALU, memória de dados e porta para escrita de registradores – só pode ser usado dentro de um

único estágio do pipeline → caso contrário, teríamos um hazard estrutural

⇒ Bug do lw apresentado → no estágio WB, como sabemos em qual registrador será escrito o dado?



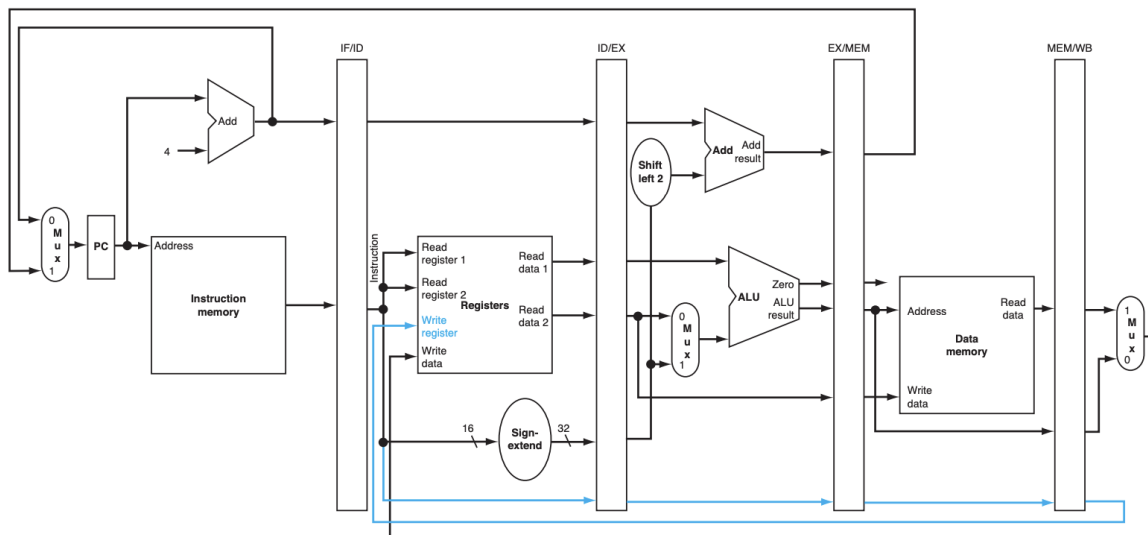
Precisamos preservar o número do registrador destino - rt no caso de load



passamos esse número para o pipe
ID/EX → EX/MEM → MEM/WB



Daí, no estágio WB podemos usar
esse valor



⇒ Representações gráficas do Pipeline:

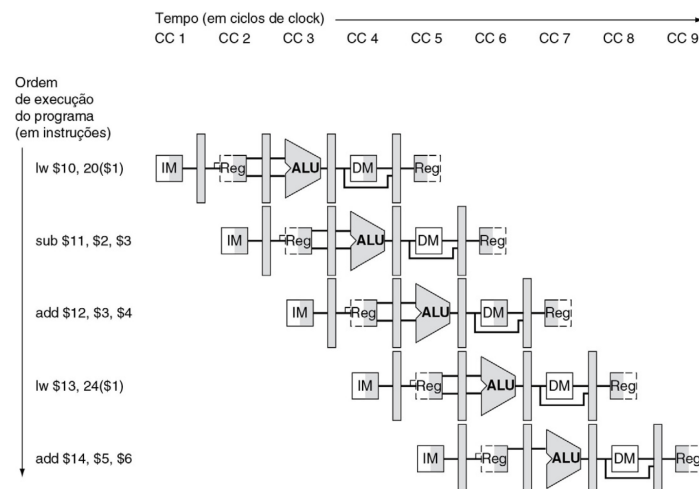


Figura 5: Exemplo de diagrama que usa os recursos físicos de cada estágio e com múltiplos ciclos de clock para uma sequência de instruções

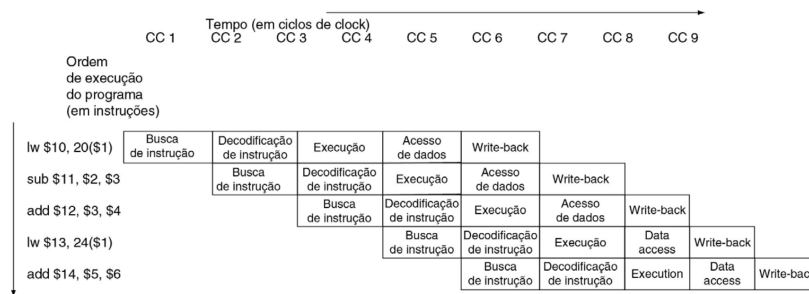


Figura 6: Exemplo de diagrama com múltiplos ciclos de clock para uma sequência de instruções usando os nomes dos estágios

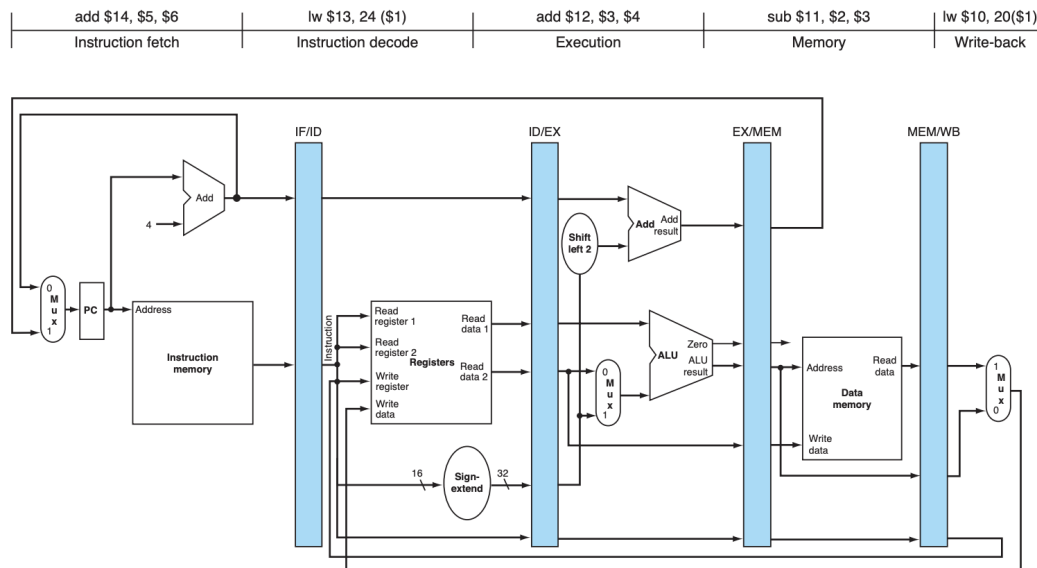


Figura 7: Exemplo de diagrama com ciclo único de clock que mostra em qual estágio cada instrução está

2.1 Controle de um pipeline

⇒ Não existem sinais de escrita para PC, IF/ID, ID/EX, EX/MEM, MEM/WB pois esses registradores são escritos a cada ciclo de clock

⇒ Devemos definir os valores de controle durante cada estágio do pipeline e, assim, podemos dividir as linhas de controle em 5 grupos (de acordo com o estágio):

1. Busca de instruções: nenhum sinal para controlar
2. Decodificação de instruções/leitura do Banco de Registradores: nenhum sinal para controlar
3. Execução/cálculo de endereço: **ALUSrc** (Escolhe a segunda entrada da ALU - dado do registradore 2 ou imediato que foi estendido), **ALUOp** (Define o que a ALU vai fazer - soma, subtração, op definida por funct, ...), **RegDst** (Escolhe qual o registrador destino da instrução - ex: para add usamos rd, enquanto que para lw usamos rt → add rd, rs, rt e lw rt,offset(rs))
4. Acesso à memória: **Branch** (Ativo em beq e é usado junto com a saída Zero da ALU a qual indica se a comparação do beq foi V ou F; se V, a saída do sinal Branch & Zero da ALU gera o sinal **PCSrc** que indica ao PC se devemos usar PC + 4 ou o endereço de desvio para escrita no PC), **MemWrite** (Ativo somente em sw) e **MemRead** (Ativo somente em lw)

5. Escrita do resultado: **MementoReg** (Escolhe qual dado enviar para o Banco de Registradores fazer a escrita no registrador destino - dado lido da memória ou resultado da ALU) e **RegWrite** (Habilita a escrita no Banco de Registradores)

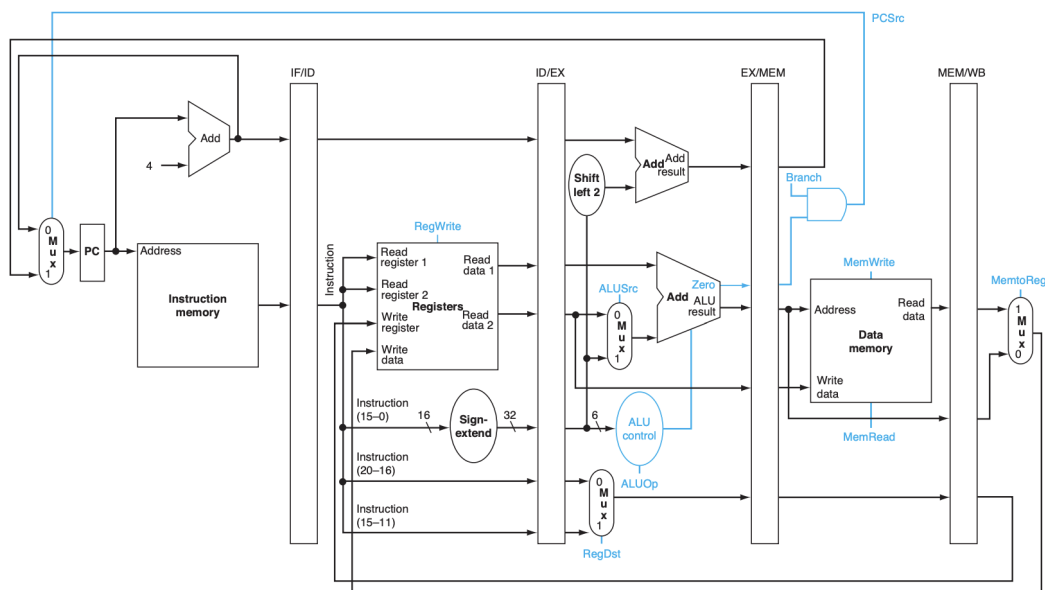


Figura 8: Datapath com sinais de controle identificados

Nome do sinal	Efeito quando inativo (0)	Efeito quando ativo (1)
RegDst	O número do registrador destino para a entrada Registrador para escrita vem do campo rt (bits 20:16).	O número do registrador destino para a entrada Registrador para escrita vem do campo rd (bits 15:11).
RegWrite	Nenhum.	O registrador na entrada Registrador para escrita é escrito com o valor na entrada Dados para escrita.
ALUSrc	O segundo operando da ALU vem da segunda saída do banco de registradores (Dados da leitura 2).	O segundo operando da ALU consiste nos 16 bits mais baixos da instrução com sinal estendido.
PCSrc	O PC é substituído pela saída do somador que calcula o valor de PC + 4.	O PC é substituído pela saída do somador que calcula o destino do desvio.
MemRead	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é colocado na saída Dados da leitura.
MemWrite	Nenhum.	O conteúdo da memória de dados designado pela entrada Endereço é substituído pelo valor na entrada Dados para escrita.
MementoReg	O valor enviado para a entrada Dados para escrita do banco de registradores vem da ALU.	O valor enviado para a entrada Dados para escrita do banco de registradores vem da memória de dados.

Figura 9: O que acontece quando cada sinal é 0 ou 1 - igual no Processador Uniciclo; **Cuidado, a tabela acima foi alterada do livro colocando os nomes dos sinais em inglês para ficar coerente com as figuras dos esquemáticos do resumo!**

Instruction	Execution/address calculation stage control lines				Memory access stage control lines			Write-back stage control lines	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memento-Reg
R-format	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

Figura 10: Valores dos sinais (0 ou 1) para cada formato de instruções especificando o estágio do pipeline

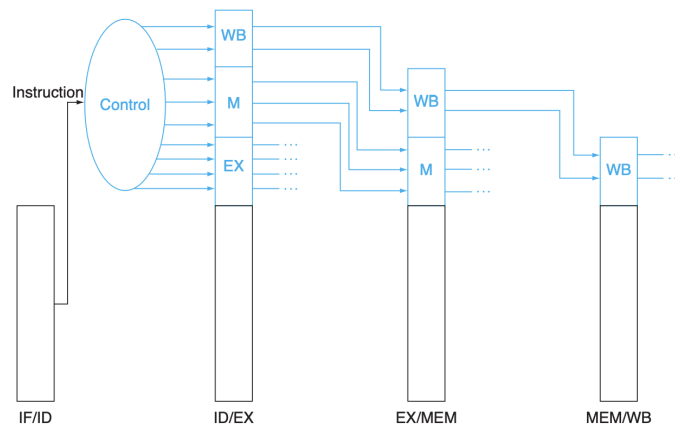


Figura 11: Os sinais de controle são usados no respectivo estágio a medida que a instrução se move pelo pipeline

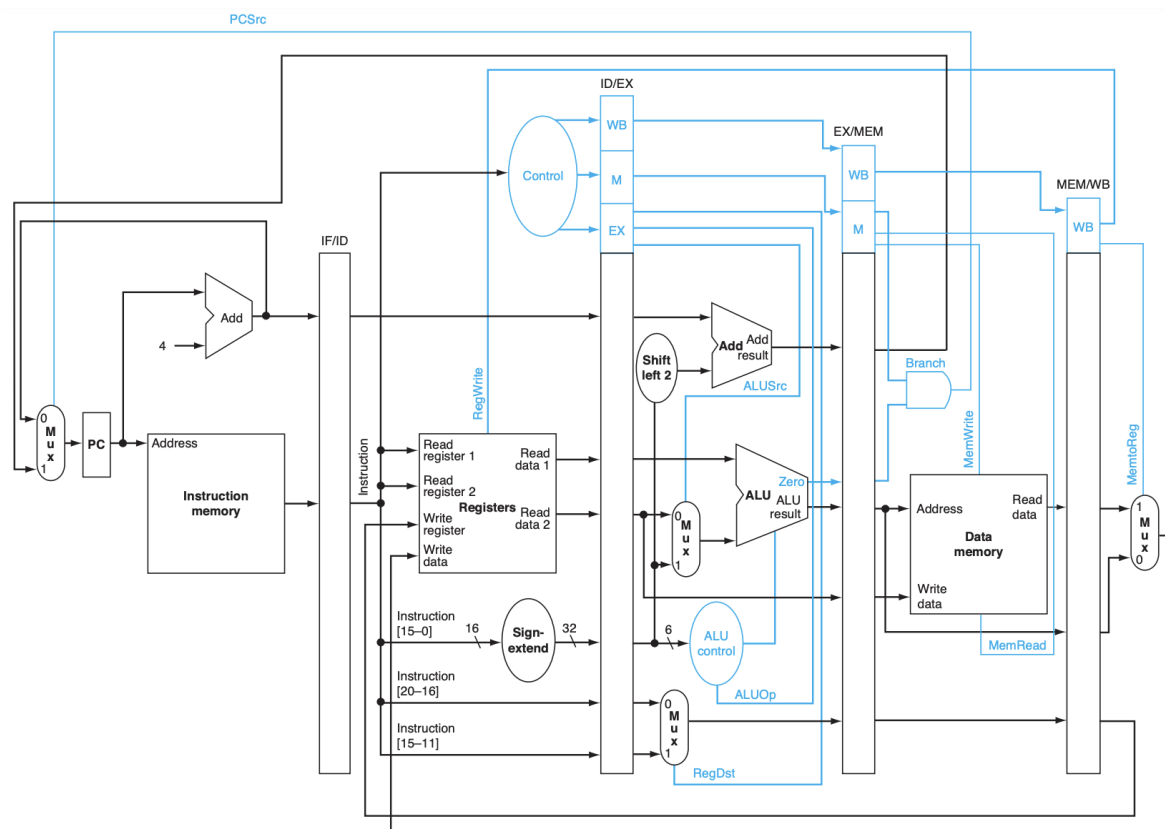


Figura 12: Caminho de dados com o controle conectado aos pipes

3 Hazards de dados: forwarding vs stalls

⇒ Potencial hazard devido a escrita e leitura no Banco de Registradores → resolvemos considerando que a escrita (instrução no estágio WB) está na primeira metade do ciclo e a leitura (instrução no estágio ID) está na segunda metade do ciclo

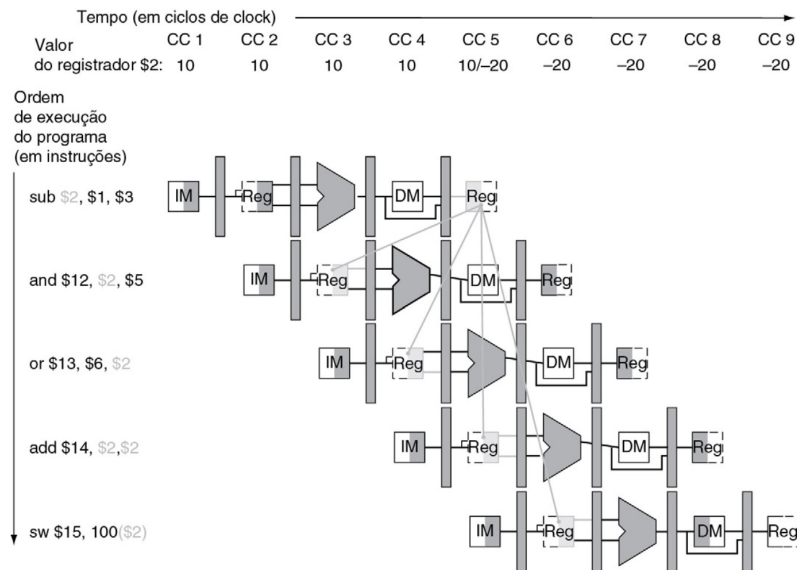


Figura 13: Exemplo de hazard de dados; para and e or não funciona o forwarding do estágio WB do sub (não é possível voltar no tempo), mas add e sw apanham o valor correto (no caso do add, como a escrita está na primeira metade do ciclo de clock e a leitura está na segunda metade, add apanha -20 corretamente em \$2 ao invés de 10)

Porém, no exemplo acima, o resultado desejado para essas duas instruções (and e or) já se encontra disponível no final do estágio EX do sub, além disso este dado é realmente necessário apenas no estágio EX de and e or



Podemos executar esse segmento sem stalls se os dados simplesmente sofrerem forwarding assim que estiverem disponíveis para quaisquer unidades que precisem deles antes que estejam disponíveis para leitura do Banco de Registradores

Podemos usar uma notação mais precisa da dependência → ex: ID/EX.RegistradorRs → refere-se ao número de um registrador cujo valor se encontra no pipe ID/EX e é o registrador 1 (rs) lido do Banco de Registradores. Assim, os dois pares de condições são:

1º par: $EX/MEM.RegistradorRd = ID/EX.RegistradorRs$
 $EX/MEM.RegistradorRd = ID/EX.RegistradorRt$



registrador que contém o dado
que deve sofrer forwarding



registrador que precisa do dado
do forwarding



ambos são o mesmo registrador, só que o usamos em instruções diferentes que geram o conflito de dados



Precisamos do dado que foi modificado em uma instrução e que já se encontra disponível em EX/MEM e fazer o forwarding dele para o estágio EX para que seja usado na instrução que depende do dado atualizado

2º par: $\text{MEM/WB.RegistradorRd} = \text{ID/EX.RegistradorRs}$
 $\text{MEM/WB.RegistradorRd} = \text{ID/EX.RegistradorRt}$



Mesma lógica da anterior, porém o forwarding saíria do pipe MEM/WB, pois a instrução que precisa do dado está dois estágios atrás, e por isso, precisamos fazer o forwarding partindo do MEM/WB

⇒ Usamos "par" pois o dado atualizado pode ser necessário a um registrador que é o rt no campo da instrução ou um registrador que é o rs no campo da instrução

Ex: `add $reg, $rs, $rt` → rs ou rt podem precisar do dado atualizado

Exemplo da figura anterior:

```

sub  $2, $1, $3
and  $12, $2, $5
or   $13, $6, $2
add  $14, $2, $2
sw   $15, 100($2)

```

I
II

I → $\text{EX/MEM.RegistradorRd} = \text{ID/EX.RegistradorRs} = \2

II → $\text{MEM/WB.RegistradorRd} = \text{ID/EX.RegistradorRt} = \2

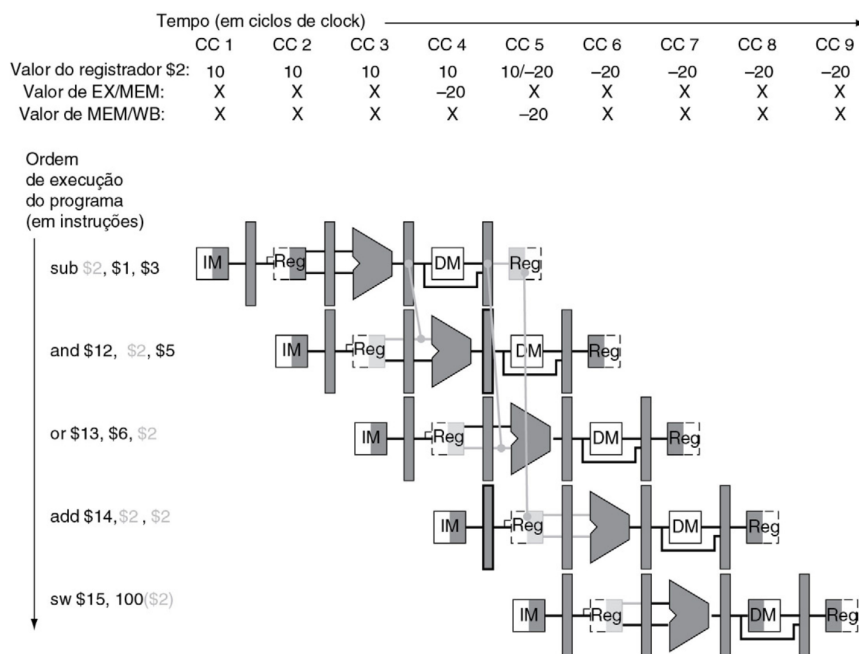


Figura 14: Forwarding correto do exemplo

⇒ Nem sempre deve haver o forwarding, por isso, devemos verificar se o sinal de RegWrite está ativo → só devemos encaminhar os dados de instruções que escrevem em registradores

⇒ Além disso, precisamos garantir que o Rd não é o registrador \$0, pois queremos evitar de enviar seu valor possivelmente diferente de 0 para o forwarding

⇒ Assim, precisaríamos ter as seguintes condições para o forwarding:

1. Hazard EX:

if(EX/MEM.RegWrite == 1 & EX/MEM.RegistradorRd ≠ 0)

Forwarding = true

2. Hazard MEM:

if(MEM/WB.RegWrite == 1 & MEM/WB.RegistradorRd ≠ 0)

Forwarding = true

Com isso, precisamos fazer o forwarding dos dados corretos ⇒ esses dados são enviados para as entradas da ALU para a instrução que precisa deles



podemos usar um mux para cada entrada da ALU com os controles apropriados (que selecionam os dados):

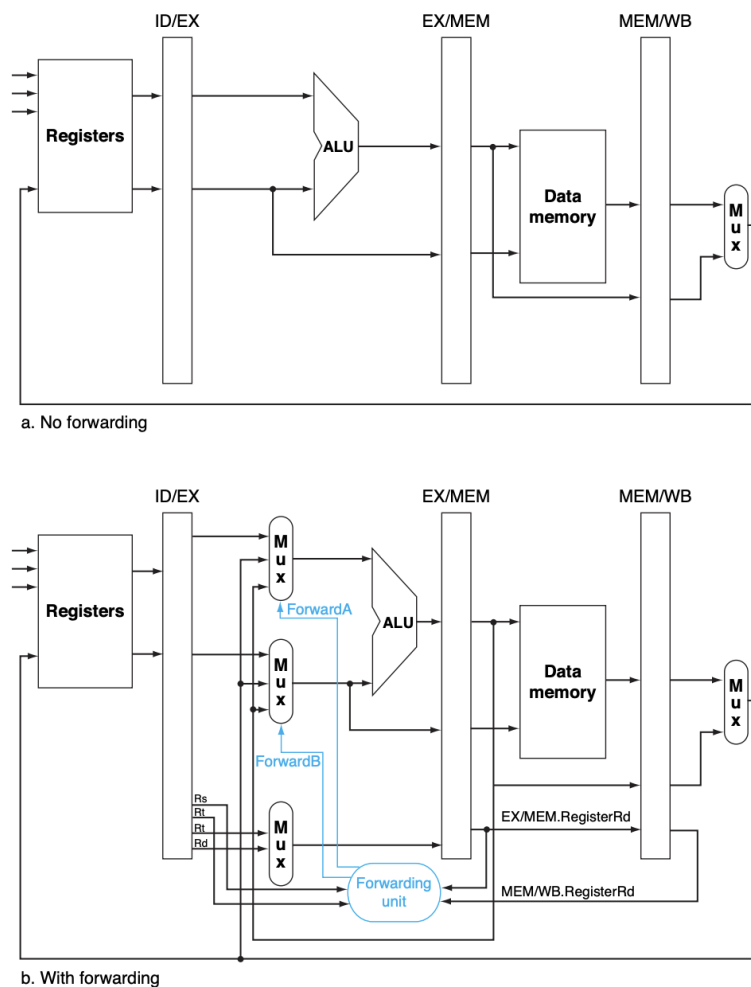


Figura 15: Adição da Unidade de Forwarding



Cada mux (A e B):

entrada 00 → dado lido do Banco de Registradores

entrada 01 → dado do MEM/WB.RegistradorRd

entrada 10 → dado do EX/MEM.RegistradorRd

⇒ Assim, temos que passar os números dos registradores operandos do estágio ID pelo pipe ID/EX

↓

Os comparamos com os números dos registradores EX/MEM.RegistradorRd e MEM/WB.RegistradorRd (que também são passados de seus respectivos pipes)

⇒ Dessa forma, temos as seguintes condições que direcionam o forwarding dos dados corretamente:

1.Hazard EX:

```
if(EX/MEM.RegWrite == 1
  & EX/MEM.RegistradorRd ≠ 0
  & EX/MEM.RegistradorRd = ID/EX.RegistradorRs)
  ForwardA = 10
```

```
if(EX/MEM.RegWrite == 1
  & EX/MEM.RegistradorRd ≠ 0
  & EX/MEM.RegistradorRd = ID/EX.RegistradorRt)
  ForwardB = 10
```

2.Hazard MEM:

```
if(MEM/WB.RegWrite == 1
  & MEM/WB.RegistradorRd ≠ 0
  & MEM/WB.RegistradorRd = ID/EX.RegistradorRs)
  ForwardA = 01
```

```
if(MEM/WB.RegWrite == 1
  & MEM/WB.RegistradorRd ≠ 0
  & MEM/WB.RegistradorRd = ID/EX.RegistradorRt)
  ForwardB = 01
```

3.Devemos pegar os valores lidos do Banco de Registradores:

else

```
ForwardA = 00
ForwardB = 00
```

Controle do Mux	Origem	Explicação
ForwardA = 00	ID/EX	O primeiro operando da ALU vem do banco de registradores.
ForwardA = 10	EX/MEM	O primeiro operando da ALU sofre forwarding do resultado anterior da ALU.
ForwardA = 01	MEM/WB	O primeiro operando da ALU sofre forwarding da memória de dados ou de um resultado anterior da ALU.
ForwardB = 00	ID/EX	O segundo operando da ALU vem do banco de registradores.
ForwardB = 10	EX/MEM	O segundo operando da ALU sofre forwarding do resultado anterior da ALU.
ForwardB = 01	MEM/WB	O segundo operando da ALU sofre forwarding da memória de dados ou de um resultado anterior da ALU.

Figura 16: Explicação valores unidade de Forwarding

⇒ Não existe hazard no estágio WB → Banco de Registradores fornece o resultado correto se a instrução no estágio ID ler o mesmo registrador escrito pela instrução no estágio WB

⇒ Complicação → hazard potencial entre o resultado da instrução no estágio WB, o resultado da instrução no estágio MEM e o operando de origem da instrução no estágio ALU (EX)

Ex: Somar um vetor de números em um único registrador → uma sequência de instruções lerá e escreverá no mesmo registrador:

```

add $1, $1, $2
add $1, $1, $3
add $1, $1, $4
. . .

```

Acima acontecem ambos os conflitos:

ID/EX.RegistradorRs = EX/MEM.RegistradorRd = \$1

ID/EX.RegistradorRs = MEM/WB.RegistradorRd = \$1

↓ Qual pegar?

O da instrução logo antes:

este é o dado mais recente/atualizado!

↓

Precisamos colocar mais condições no "hazard MEM" para evitar que seja encaminhado o dado do pipe MEM/WB da instrução que está no estágio WB pois nesse caso devemos encaminhar o dado do pipe EX/MEM da instrução que está no estágio MEM já que esta é a instrução logo antes

2. Hazard MEM:

```

if(MEM/WB.RegWrite == 1
  & MEM/WB.RegistradorRd ≠ 0
  & NOT ((EX/MEM.RegWrite == 1)
    & (EX/MEM.RegistradorRd ≠ 0)
    & (EX/MEM.RegistradorRd ≠ ID/EX.RegistradorRs))
  & MEM/WB.RegistradorRd = ID/EX.RegistradorRs)
  ForwardB = 01

```

```

if(MEM/WB.RegWrite == 1
  & MEM/WB.RegistradorRd ≠ 0
  & NOT ((EX/MEM.RegWrite == 1) *
    & (EX/MEM.RegistradorRd ≠ 0) **
    & (EX/MEM.RegistradorRd ≠ ID/EX.RegistradorRt )) ***)
  & MEM/WB.RegistradorRd = ID/EX.RegistradorRt)
  ForwardB = 01

```

Para garantir o encaminhamento correto no caso mencionado, não podemos ter uma instrução no estágio MEM que:

*Vai escrever em um registrador e

**Não tem como destino o registrador \$zero e

***Tem como destino de escrita um registrador que é usado na instrução que está em EX

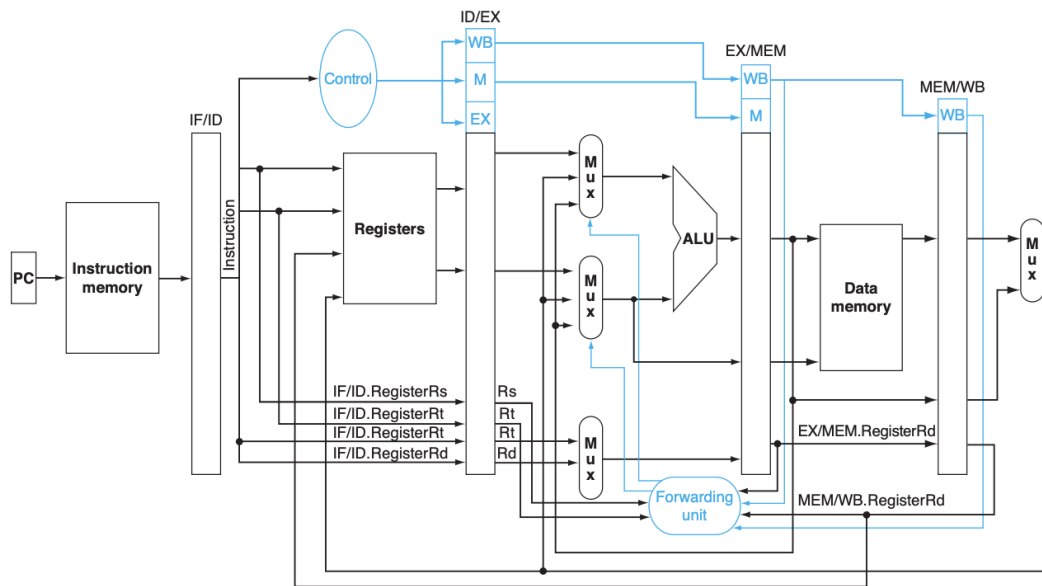


Figura 17: Adições necessárias para a "Unidade de Forwarding"

↓

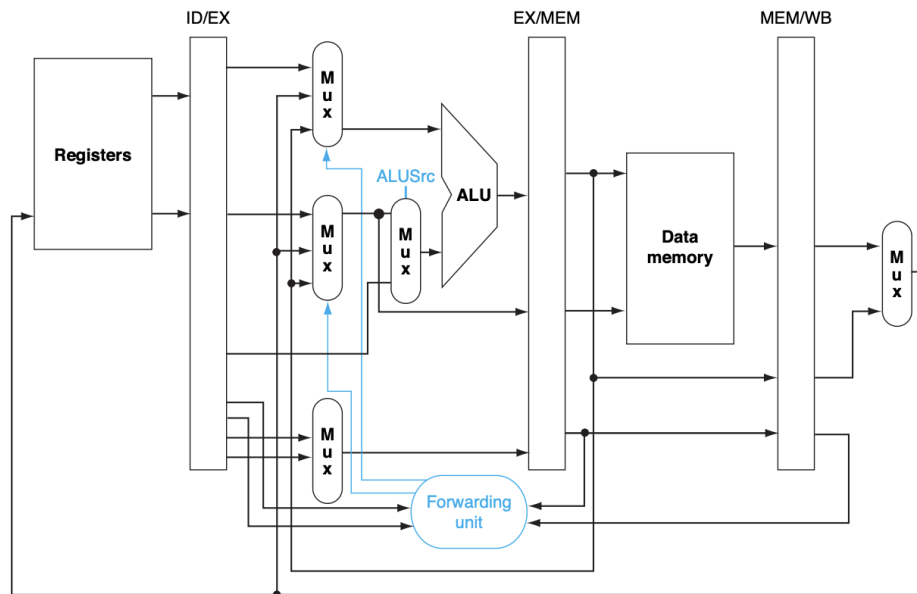


Figura 18: É preciso acrescentar um mux 2:1 na entrada B da ALU (este já estava lá antes dos acréscimos feitos na figura anterior - a segunda entrada deste mux é o imediato estendido!)

3.1 Hazards de dados e stalls

⇒ Caso em que o forwarding não resolve: quando uma instrução tenta ler um registrador após uma instrução lw que escreve no mesmo registrador

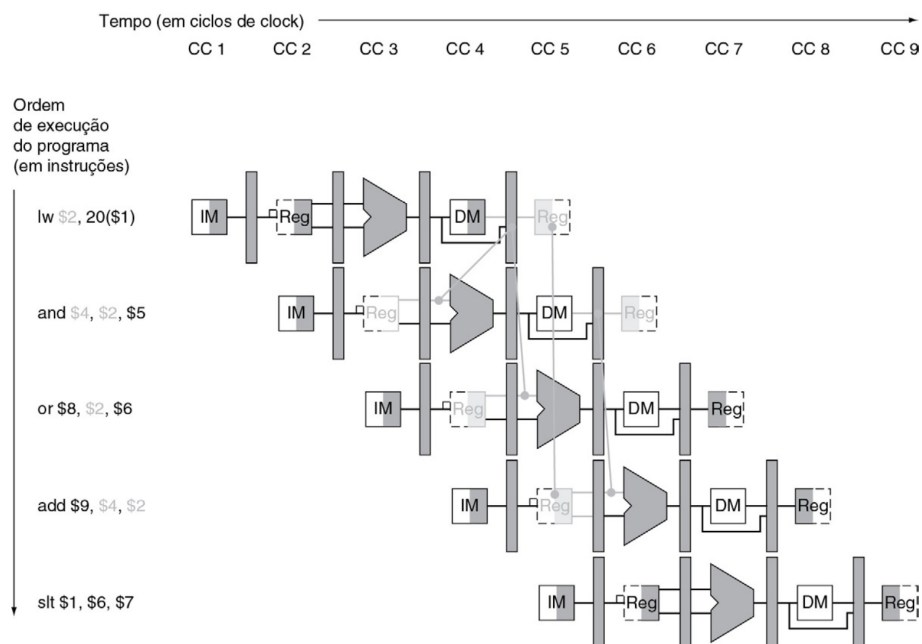


Figura 19: Hazard entre lw e and devido ao \$2 (teríamos que fazer um forwarding que volta no tempo) pois os dados são lidos da memória no lw enquanto o add está usando a ALU

↓ Solução

Stall/Bolha

↓

Para isso precisamos de uma
unidade de detecção de hazard

A unidade de detecção de hazard opera no estágio ID → pode inserir o stall entre o lw e seu uso

```
if(ID/EX.MemRead == 1
  & (ID/EX.RegistradorRt = IF/ID.RegistradorRs
    OR ID/EX.RegistradorRt = IF/ID.RegistradorRt))
  Stall no pipeline
```

O registrador no qual armazenamos o dado em lw é o rt → lw rt, offset(rs)

Assim, analisamos se o rt/rs da instrução que vem após o lw é igual ao rt da instrução lw → se sim, quer dizer que estamos tentando usar um dado que ainda não terminou de ser carregado da memória

Depois do stall de 1 ciclo, a lógica do forwarding pode lidar com a dependência de dados e prosseguir a execução → lembre-se que a notação "RegistradorRd" se refere ao registrador de destino, que é rd em instruções do tipo R e rt em lw

Se a instrução no estágio ID sofrer um stall, então a instrução no estágio IF também deve → impedimos que PC e o pipe IF sejam alterados desativando seus sinais de escrita → **PCWrite = 0** e **IF/ID.Write = 0**

Desde que esses dois registradores (PC e IF/ID) sejam preservados, a instrução no pipe IF continuará a ser lida usando o mesmo PC, e os registradores no estágio ID continuarão a ser lidos usando os mesmos campos de

instrução no registrador de pipeline IF/ID

Devido ao stall, precisamos inserir uma instrução que não executa nada nos próximos estágios no pipeline (EX, MEM e WB): **nop**

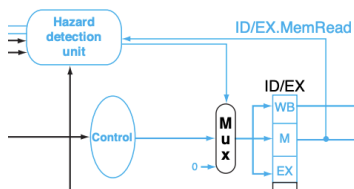
Retornando à nossa analogia, é como se você reiniciasse a lavadora com as mesmas roupas e deixasse a secadora continuando a trabalhar vazia → naturalmente, assim como a secadora, a metade do pipeline que começa com o estágio EX precisa estar fazendo algo; o que ela está fazendo é executar instruções que não têm efeito algum, nops

Como inserir um nop?

Desativamos todos os 9 sinais de controle nos estágios EX, MEM e WB: ALUSrc, ALUOp, RegDst, Branch, PCSrc, MemWrite, MemRead, MemtoReg e RegWrite

Como desativá-los?

mux controlado pela unidade de detecção de hazard seleciona 0 para todos os sinais:



↓

Esses valores de controle 0 são filtrados adiante em cada ciclo de clock com o efeito correto: nenhum registrador ou memória serão modificados se os valores forem todos 0

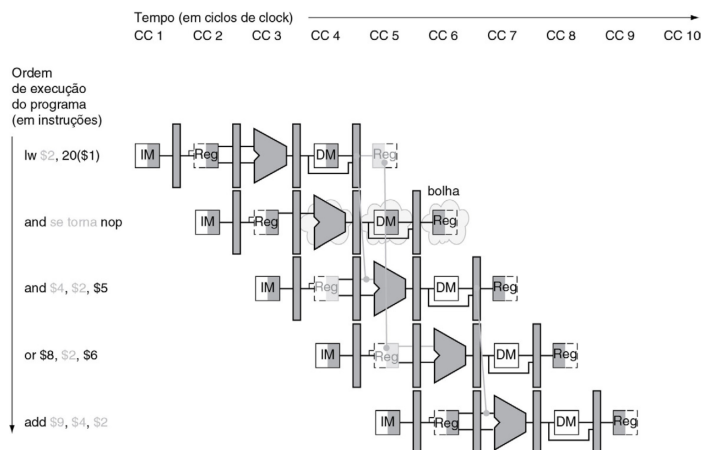
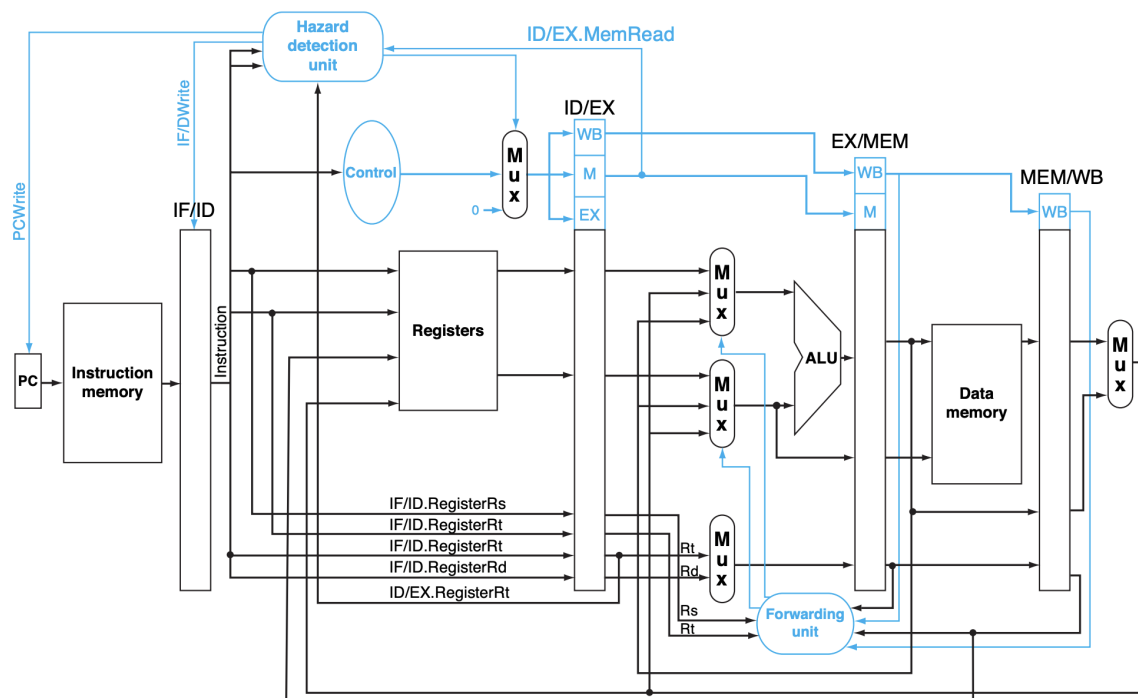


Figura 20: O hazard força as instruções and e or a repetirem o que haviam feito 1 ciclo antes: and lê os registradores e os decodifica, e or é apanhado novamente da memória de instruções



Unidade de Forwarding → controla os mux das entradas da ALU a fim de substituir o valor de um registrador de uso geral pelo valor do registrador de pipeline apropriado no caso de um forwarding

Unidade de detecção de Hazard → controla a escrita dos registradores PC e IF/ID mais o mux que escolhe entre os valores de controle reais e 0 para todos os sinais (inserção do nop por consequência do stall)

4 Hazards de controle

⇒ Uma instrução precisa ser buscada a cada ciclo de clock para sustentar o pipeline → em nosso projeto a decisão de desvio não ocorre até o estágio MEM, como vemos na figura 12

⇒ Este atraso para determinar a instrução própria a ser buscada é o hazard de controle

⇒ **Considere que o desvio não foi tomado** → continuar no fluxo sequencial das instruções e se o desvio for tomado as instruções que estão sendo buscadas e decodificadas precisam ser descartadas

Se os desvios não são tomados na metade das vezes, e se custar pouco descartar as instruções, essa otimização reduz ao meio o custo dos hazards de controle

Para descartar as instruções, simplesmente alteramos os valores de controle para 0 como no hazard de dados no caso do lw

A diferença é que também precisamos alterar as três instruções nos estágios IF, ID e EX quando o desvio atingir o estágio MEM

Descartar instruções significa que precisamos dar **flush** nas instruções nos estágios IF, ID e EX como ocorre no exemplo abaixo:

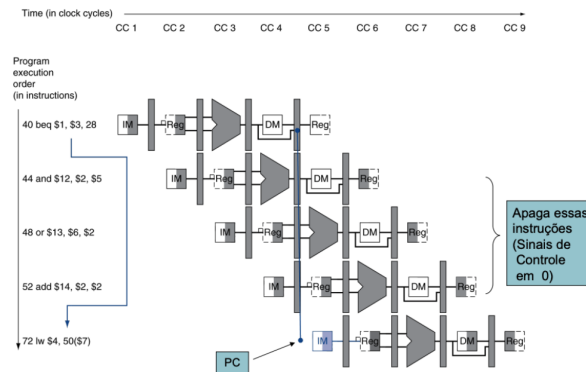


Figura 21: Como a instrução branch decide se deve desviar no estágio MEM, as 3 instruções sequenciais que seguem o beq serão buscadas e iniciarão sua execução → essas instruções devem ser transformadas em nop no caso do desvio ser tomado

4.1 Reduzindo o atraso dos desvios

⇒ Consideramos até aqui que o próximo PC para um desvio é selecionado no estágio MEM, mas, se movermos a execução do desvio para um estágio anterior de pipeline, então menos instruções precisam sofrer flush

⇒ Muitos desvios contam apenas com alguns testes simples (igualdade ou sinal) → não exigem uma operação completa da ALU (podem ser feitos com portas lógicas)

⇒ Levar a decisão de desvio para cima exige que 2 ações ocorram mais cedo:

1. Calcular o endereço de destino do desvio
2. Avaliar a decisão do desvio

1 → Já temos o valor de PC e o campo imediato (valor do desvio) no pipe IF/ID, de modo que só precisamos mover o somador do desvio do estágio EX para o estágio ID → naturalmente, o cálculo do endereço de destino do desvio será realizado para todas as instruções, mas só será usado quando for necessário

2 → Para beq comparamos os dois registradores lidos durante o estágio ID para ver se são iguais (=)

⇒ Mover o teste do desvio para ID implica em hardware adicional de forwarding e detecção de hazard

↓

Um desvio dependente de um resultado ainda no pipeline precisará funcionar corretamente com esta otimização

⇒ A fim de implementar beq e bne, teremos de fazer um forwarding dos resultados para a lógica do teste de igualdade que opera durante o estágio ID

⇒ Fatores que complicam o procedimento:

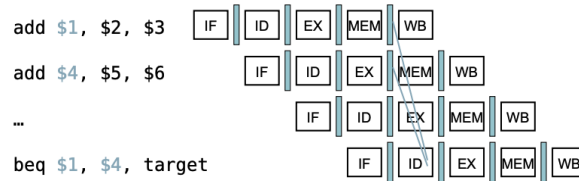
1) No estágio ID temos de:

- Decodificar a instrução
- Decidir se um forwarding para a unidade de igualdade é necessário
- Completar a comparação de igualdade de modo que, se a instrução for um desvio, possamos atribuir ao PC o endereço de destino do desvio

O forwarding para os operandos dos desvios foi tratado anteriormente pela lógica de forwarding da

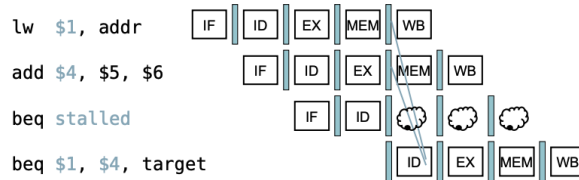
ALU, mas a introdução da unidade de teste de igualdade no estágio ID exigirá uma nova lógica de forwarding!

Os operandos-fontes de um desvio que sofreram forwarding podem vir dos latches dos pipes EX/MEM ou MEM/WB, como no exemplo abaixo (Se um dos registradores a serem comparados for o destino de uma instrução tipo-R anterior a 2 ou 3 clocks de distância):

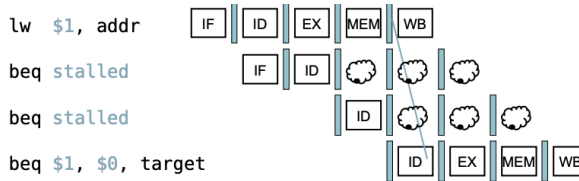


2) Como os valores em uma comparação de desvio são necessários durante o estágio ID, mas podem ser produzidos mais adiante do tempo, é possível que ocorra um hazard de dados e um stall seja necessário

Se uma instrução do tipo R está no estágio EX imediatamente antes de um desvio ou uma de lw no estágio MEM (2 instruções antes de um desvio) produzir um dos operandos (resultado da ALU ou dado lido da Memória de Dados) para comparação no desvio, um stall será exigido, já que o estágio EX para a instrução que usa a ALU (tipo R) ou o estágio MEM que usa a Memória (lw) ocorrerá depois do ciclo do estágio ID da instrução de desvio, como no exemplo abaixo:



Se um lw for imediatamente seguido por um desvio condicional que está no resultado do lw, dois ciclos de stall serão necessários, pois o resultado do lw aparece no final de cada ciclo MEM, mas é necessário no início do estágio ID da instrução de desvio como no exemplo abaixo:



⇒ Mesmo com os obstáculos, mover a execução do desvio para o estágio ID é uma melhoria, pois reduz a penalidade de um desvio a apenas 1 instrução se o desvio for tomado, a saber, aquela sendo buscada atualmente

⇒ Como fazer o flush das instruções no estágio IF? (necessário no caso do desvio ser tomado)

■ Linha de controle IF.Flush → zera o campo de instrução do pipe IF/ID

↓

transforma a instrução buscada
em um nop → não possui ação
e não muda nenhum estado

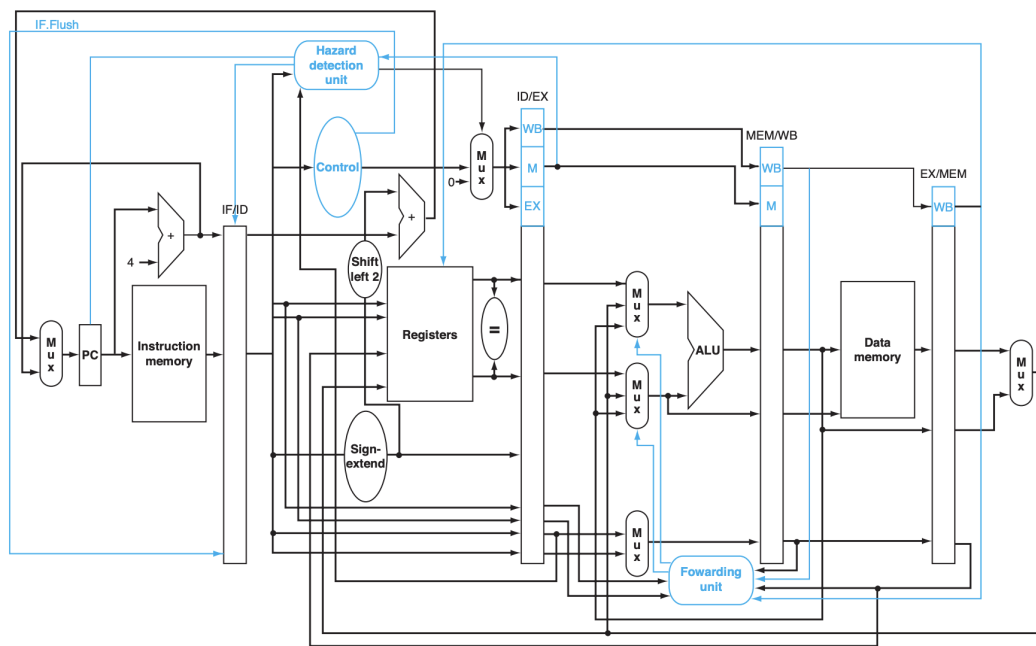


Figura 22: Observe que essa é uma figura estilizada, em vez de um caminho de dados detalhado, de modo que não contém o mux OrigALU e nem os controles dos multiplexadores da figura 12

Acima temos os acréscimos:

- Mux que seleciona entre $PC + 4$ ou o endereço de desvio
- Linha IF.Flush → usamos para inserir um stall no IF (necessário no caso do desvio ser tomado)
- Unidade de soma
- Extensão de sinal para os bits do imediato e o deslocamento de 2 bits a esquerda para estes mesmos bits do imediato → este valor é uma das extradas da unidade de soma (outro acréscimo) e a outra entrada é $PC + 4$ → aqui é feito o cálculo do endereço de desvio que é passado para PC (usamos o MUX adicionado para selecionar este valor no caso de desvio)
- Unidade de comparação (=) dos valores lidos dos registradores → controla o mux do PC

⇒ Exemplo:

```

36:  sub  $10, $4, $8
40:  beq  $1,  $3, 7
44:  and  $12, $2, $5
48:  or   $13, $2, $6
52:  add  $14, $4, $2
56:  slt  $15, $6, $7
...
72:  lw   $4, 50($7) // 72 = 44 + 7x4

```

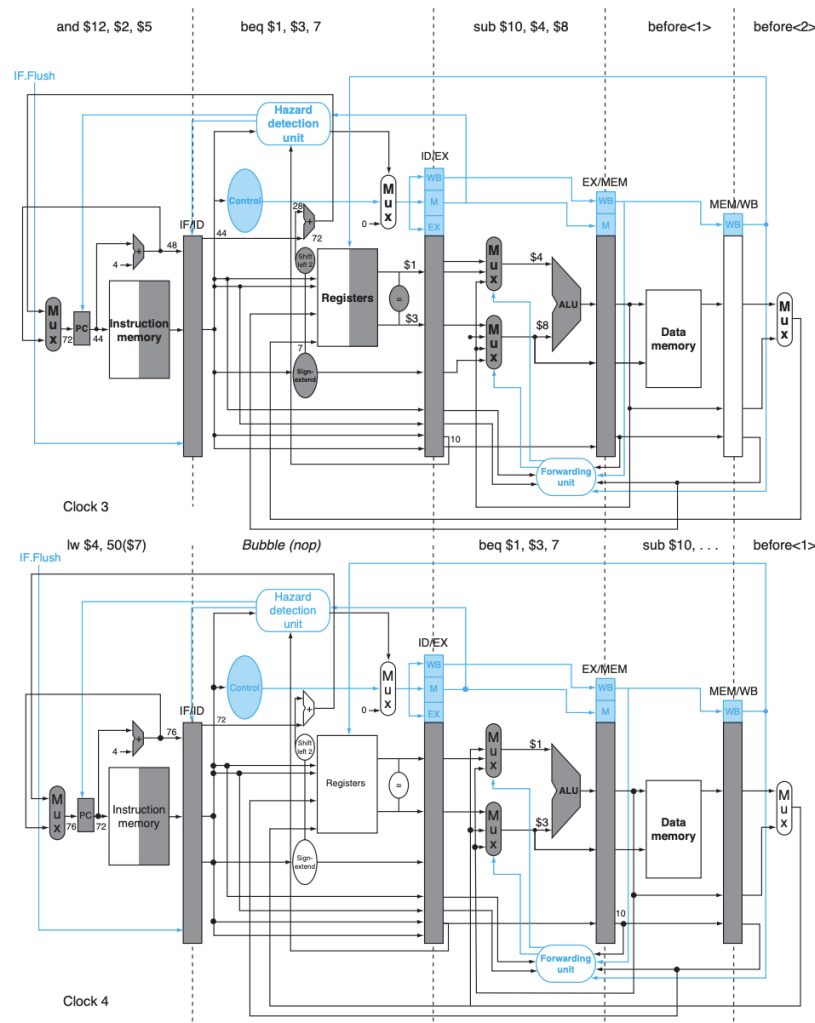


Figura 23: Caminho de dados do exemplo acima considerando a decisão de desvio no estágio ID

4.2 Previsão dinâmica de desvios

⇒ Supor que um desvio não será tomado:

- Fazemos um flush quando estivermos errados
- Para o pipeline simples, com cinco estágios, essa técnica, possivelmente acoplada com a previsão baseada no compilador, deverá ser adequada
- Mas com pipelines mais profundos e superescalares (vários componentes paralelos como ULA's), a penalidade do desvio aumenta quando medida em ciclos de clock

⇒ Previsão dinâmica de desvios → pesquisar o endereço da instrução para ver se um desvio foi tomado na última vez que essa instrução foi executada e, se foi, começar a buscar novas instruções a partir do mesmo lugar da última vez

⇒ Podemos implementar com um **buffer de previsão de desvios** → pequena memória* indexada pelos endereços de desvios recentemente executados

*contém 1 bit que armazena resultado da condição (tomado/não tomado)

⇒ Ao executar desvio:

Checar o buffer e assumir o mesmo resultado

Buscar próxima instrução ou destino do desvio (dependendo do buffer)

A previsão é apenas um palpite: se estiver errado, as instruções previstas incorretamente são excluídas, o bit de previsão é invertido e a sequência apropriada é buscada e executada

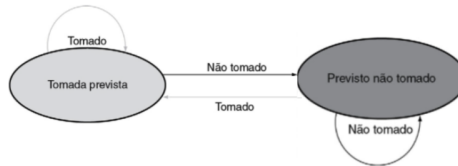


Figura 24: Esquema de previsão de 1-bit

⇒ Exemplo mostrando o problema de desempenho em loops do esquema de previsão de 1 bit:

Considere um desvio de loop que se desvia nove vezes seguidas, depois não é tomado uma vez. Qual é a exatidão da previsão para esse desvio, supondo que o bit de previsão para o desvio permaneça no buffer de previsão?

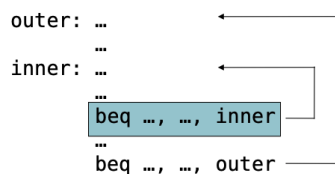
O comportamento da previsão de estado fixo fará uma previsão errada na primeira e última iterações do loop:

O erro de previsão na última iteração é inevitável, pois o bit de previsão dirá “tomado”, já que o desvio foi tomado nove vezes seguidas nesse ponto

O erro de previsão na primeira iteração acontece porque o bit é invertido na execução anterior da última iteração do loop, pois o desvio não foi tomado nessa iteração final

Assim, a exatidão da previsão para esse desvio tomado 90% do tempo é apenas de 80% (2 previsões incorretas contra 8 corretas)

⇒ Assim, o predictor de 1-bit apresenta essa deficiência: desvios de loops são previstos incorretamente 2 vezes!

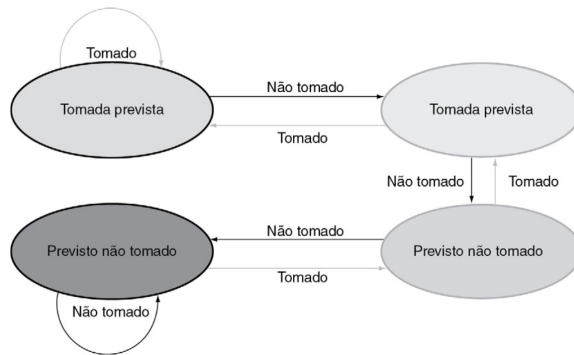


Prevê erradamente como tomado na última iteração do laço de repetição (inevitável)

Depois, prevê erradamente como não tomado, na 1ª iteração do laço de repetição para a próxima rodada do laço

⇒ Podemos usar o **esquema de previsão de 2 bits**:

uma previsão precisa estar errada duas vezes antes de ficar alterada



⇒ Calculando o destino do desvio

Um preditor de desvios nos diz se um desvio é tomado ou não, mas ainda exige o cálculo do destino do desvio → no pipeline de cinco estágios, esse cálculo leva 1 ciclo significando que os desvios tomados terão uma penalidade de 1 ciclo

Uma técnica para eliminar essa penalidade é usar uma cache para manter o contador de programa (PC) de destino ou a instrução de destino, usando um **buffer de destino de desvios** → se estiver no buffer e for previsto como tomado, pode buscar a próxima instrução imediatamente

4.3 Escalonamento com delay slot do desvio

⇒ Os compiladores e os montadores tentam colocar uma instrução que sempre executa após o desvio no delay slot do desvio

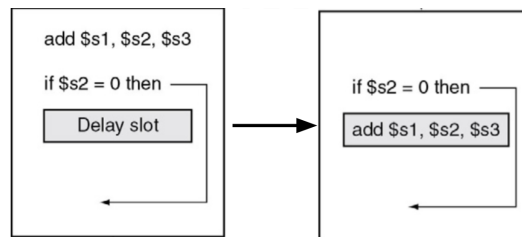
⇒ A tarefa do software é tornar as instruções sucessoras válidas e úteis

⇒ Limitações:

1. As restrições do Banco de Registradores e as instruções escalonadas nos Delay Slots (ex: não pode depender nem afetar a condição testada)
2. Nossa capacidade de prever durante a compilação se um desvio provavelmente será tomado ou não

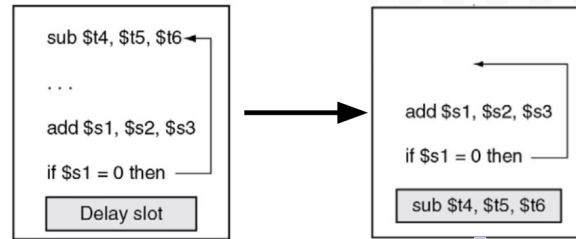
⇒ Exemplos:

a. Do anterior



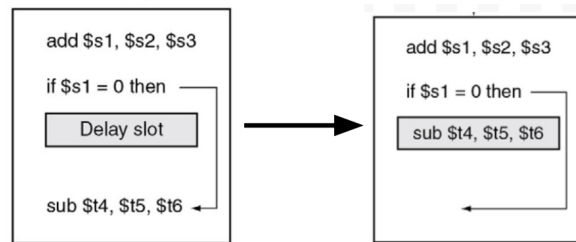
- O delay slot é escalonado com uma instrução independente antes do desvio
- Melhor opção, mas nem sempre é possível
- Quando (a) não funciona, usamos as estratégias (b) e (c)

b. Do destino



- O uso de \$s1 como destino na primeira condição impede que a instrução add seja movida para o delay slot
- O delay slot de desvio é escalonado a partir do destino do desvio; normalmente a instrução de destino precisará ser copiada, pois pode ser alcançada por outro caminho
- Essa estratégia é preferida quando o desvio é tomado com alta probabilidade, como em um desvio de um loop

c. Do fall-through



- O desvio pode ser escalonado a partir da sequência não tomada como acima
- O uso de \$s1 como destino na primeira condição também impede que a instrução add seja movida para o delay slot

⇒ Essa otimização usando sub só é válida em (b) e (c) se for "OK" executar a instrução sub quando o desvio seguir na direção inesperada

⇒ Por "OK" queremos dizer que o trabalho é desperdiçado, mas o programa ainda será executado corretamente → esse é o caso, por exemplo, se \$t4 fosse um registrador temporário não utilizado quando o desvio entrasse na direção inesperada

5 Exceções

⇒ Exceções e interrupções → eventos diferentes dos desvios ou saltos que mudam o fluxo normal de execução da instrução

⇒ Indicam qualquer mudança inesperada no fluxo de controle

⇒ No MIPS, o termo exceção é usado para indicar qualquer mudança inesperada no fluxo de controle, sem distinguir se a causa é interna ou externa; usamos o termo interrupção apenas quando o evento é causado externamente

Tipo de evento	De onde?	Terminologia MIPS
Solicitação de dispositivo de E/S	Externa	Interrupção
Chamar o sistema operacional do programa do usuário	Interna	Exceção
Overflow aritmético	Interna	Exceção
Usar uma instrução indefinida	Interna	Exceção
Defeitos do hardware	Ambos	Exceção ou interrupção

5.1 Como tratar exceções no MIPS

⇒ No MIPS, exceções são gerenciadas por um Coprocessador de Controle do Sistema (CP0)

⇒ Em nossa implementação temos duas possíveis:

- Instrução indefinida
- Overflow aritmético

⇒ Quando ocorre uma exceção, o processador deve:

- Salvar o endereço da instrução causadora no PC de exceção (EPC)
- Transferir o controle para o sistema operacional em algum endereço especificado

⇒ O sistema operacional pode então tomar a ação apropriada

⇒ Depois, o sistema operacional pode terminar o programa ou pode continuar sua execução, usando o EPC para determinar onde reiniciar a execução

⇒ Métodos para comunicar o motivo da exceção:

- Registrador de status (Cause)
- Interrupções vetorizadas → nelas o endereço do tratador de exceção ao qual o controle é transferido é determinado pela causa da exceção, por exemplo:

Tipo de exceção	Endereço do vetor de exceção (em _{hex})
Instrução indefinida	8000 0000 _{hex}
Overflow aritmético	8000 0180 _{hex}

⇒ Precisamos adicionar ao caminho de dados 2 registradores:

- **EPC** → mantém o endereço da instrução afetada ⇒ 32 bits
- **Cause** → registra a causa da exceção

↓

Vamos assumir que tem 1 bit:

- 0 → opcode indefinido
- 1 → overflow

5.2 Exceções em uma implementação em pipeline

⇒ Pipeline as trata como uma forma de Hazard de controle

⇒ A exceção causa a desativação das linhas de controle

⇒ Quando lidamos com um desvio mal previsto, damos um flush na instrução no estágio IF, transformando-a em um nop, como vimos anteriormente

⇒ Para dar flush nas instruções no estágio ID, usamos o mux já presente no estágio ID que zera os sinais de controle para ocasionar stalls → um novo sinal de controle, ID.Flush, realiza um OR com o sinal de stall da Unidade de Detecção de Hazards a fim de dar flush durante o estágio ID

⇒ Para dar Flush nas instruções no estágio EX, usamos o EX.Flush, que faz com que novos mux zerem as linhas de controle

⇒ Devemos salvar o endereço da instrução problemática no EPC (na realidade, salvamos o endereço + 4: PC + 4) e a causa da exceção no registrador Cause

⇒ Para começar a buscar o endereço da primeira instrução da rotina de exceção simplesmente acrescentamos uma entrada ao mux do PC com este endereço - no exemplo 8000 0180₁₆ para overflow aritmético

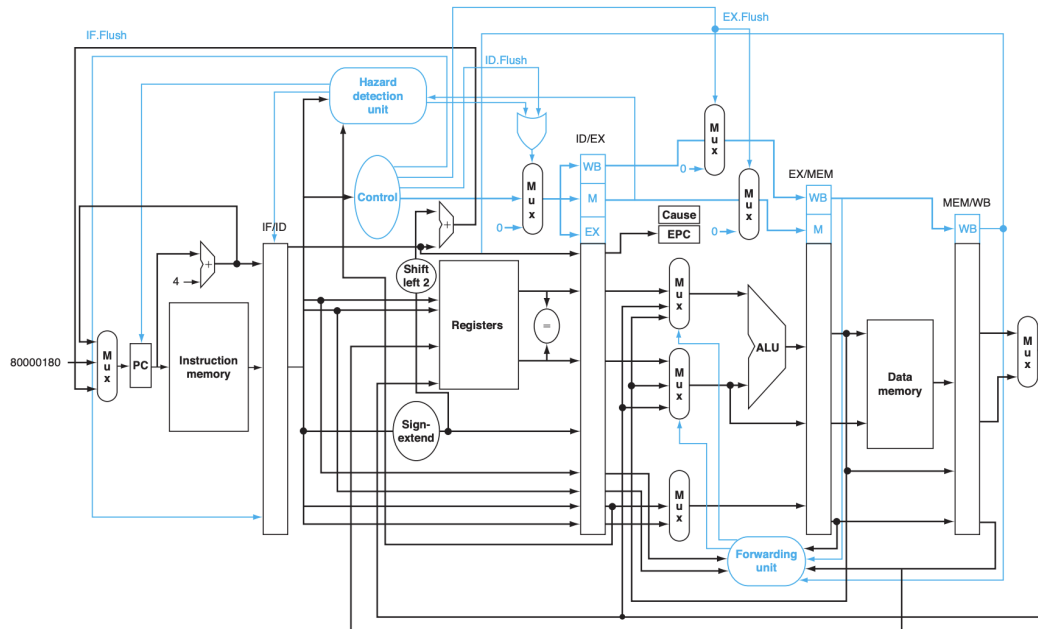


Figura 25: Caminho de dados com os acréscimos mencionados acima

Acréscimos

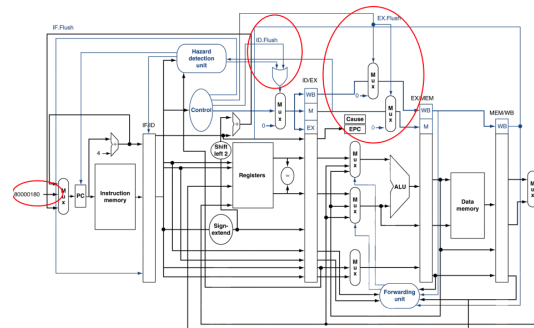
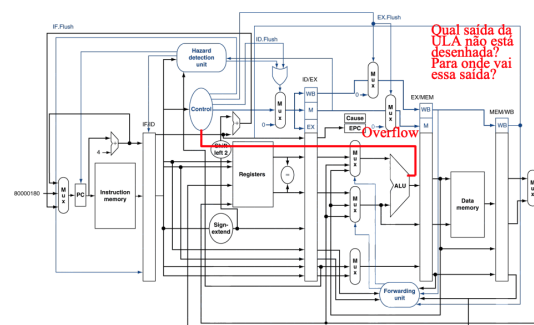


Figura 26: Por quê não existem os sinais MEM.Flush e WB.Flush? Neste exemplo, estamos tratando uma exceção ocorrida no 3º estágio!



⇒ Devido ao planejamento do projeto, a exceção de overflow é detectada durante o estágio EX → podemos usar o EX.Flush para impedir que a instrução no estágio EX escreva seu resultado no estágio WB

⇒ Após o tratador da exceção ter executado as correções, muitas instruções exigem que, por fim, completemos a instrução que causou a exceção como se ela fosse executada normalmente → dar flush na instrução e reiniciá-la desde o início usando o EPC após a exceção ser tratada (como salvamos PC + 4 no EPC, a rotina de tratamento da exceção primeiro deve subtrair 4 do valor salvo)

⇒ Se o tratador não corrigir: termina o programa e reporta erro usando EPC, Registrador de causa, ...

⇒ Exemplo:

```

40hexa sub    $11, $2, $4
44hexa and    $12, $2, $5
48hexa or     $13, $2, $6
4Chexa add    $1, $2, $1
50hexa slt    $15, $6, $7
54hexa lw     $16, 50($7)
  
```

Figura 27: add causa exceção de overflow

```

80000180hexa SW      $26, 1000($0)
80000184hexa SW      $27, 1004($0)
  
```

Figura 28: Instruções a serem invocadas na exceção

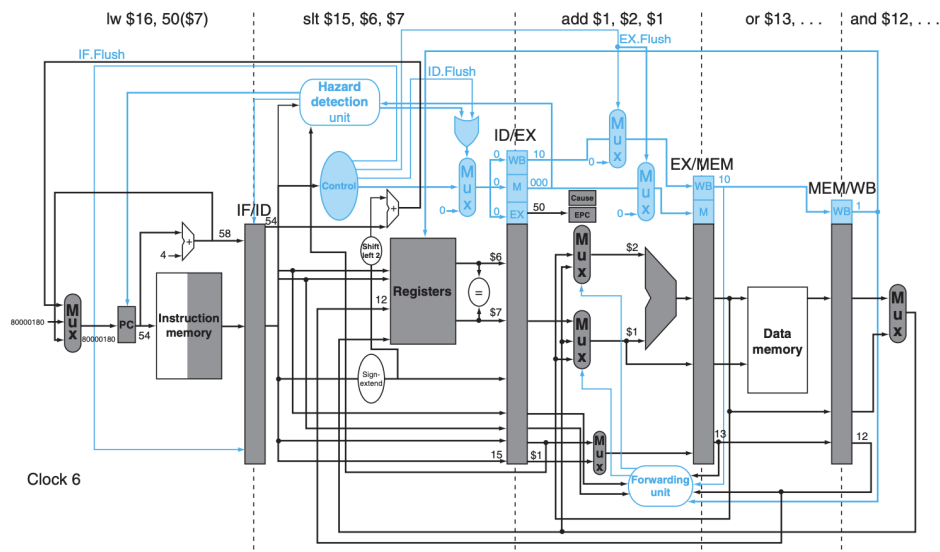


Figura 29: Overflow do add detectado no estágio EX (O sinal de overflow da ALU é uma entrada para a unidade de controle - não está na figura acima) ⇔ add e as instruções seguintes ao add (que estão em estágios anteriores no pipeline) sofrem flush (ativamos todos os sinais de flush que desativam os valores de controle para o add e instruções seguintes ao add) ⇔ O endereço 50 (próxima instrução após o add) é salvo no EPC ⇔ and e or (instruções anteriores ao add) ainda completam

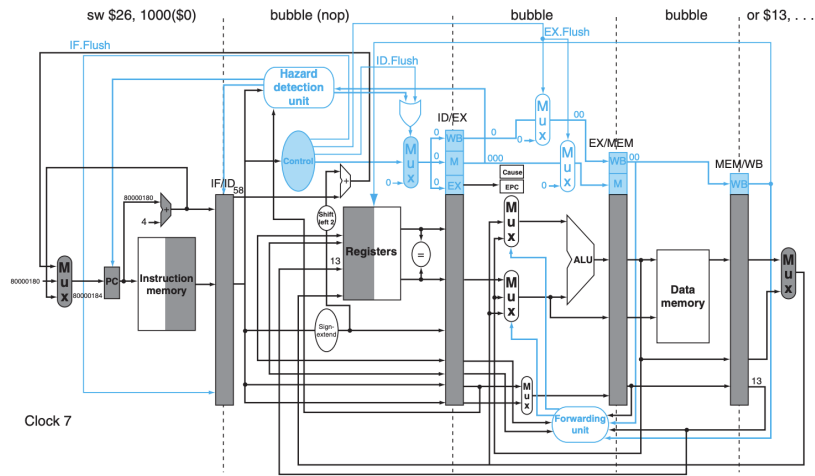


Figura 30: 8000 0180 é forçado para o PC ⇔ A primeira instrução do código de exceção é buscada e executada (primeiro sw)

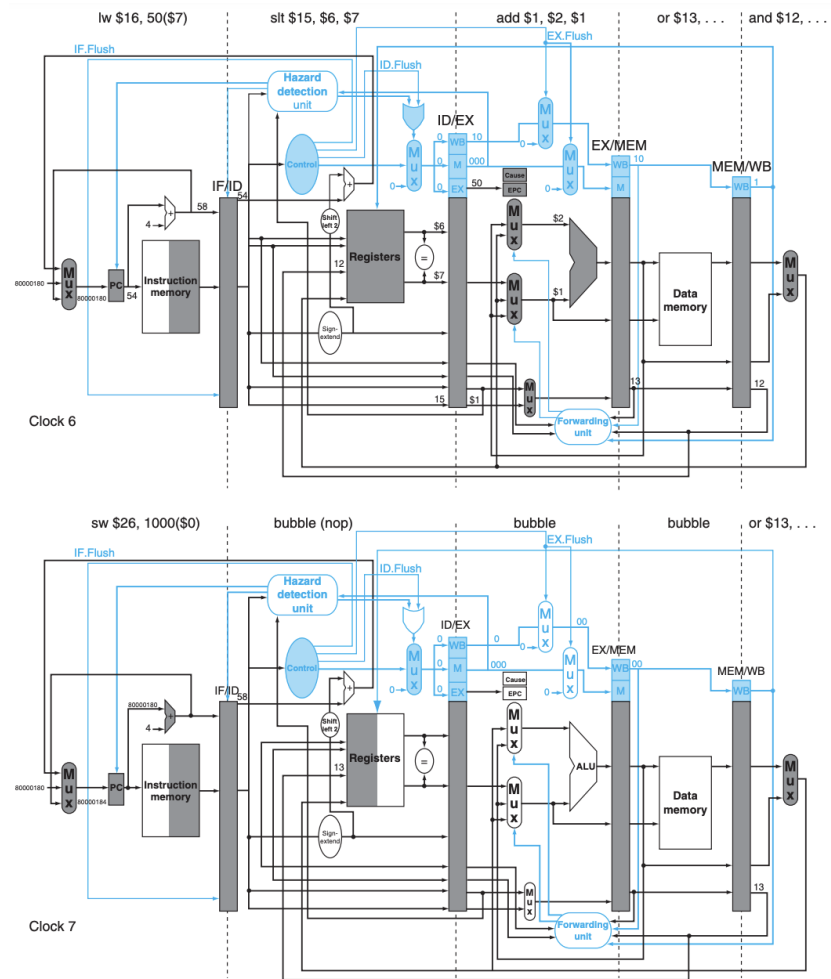


Figura 31: Visão completa do exemplo dos 2 últimos datapaths analisados

⇒ Mencionamos cinco exemplos de exceções na tabela 5 → com cinco instruções ativas em qualquer ciclo de clock, temos o desafio de associar uma exceção à instrução apropriada

Além do mais, várias exceções podem ocorrer simultaneamente em um único ciclo de clock → Exceções Múltiplas

A solução é priorizar as exceções de modo que seja fácil determinar qual será atendida primeiro → Na maioria das implementações MIPS, o hardware ordena as exceções de modo que a instrução mais antiga seja interrompida → Exceções Precisas

O MIPS e a grande maioria dos computadores de hoje admitem Interrupções Precisas ou Exceções Precisas → um motivo é para dar suporte à memória virtual (falha no acesso à cache ou à memória)

⇒ A dificuldade de sempre associar a exceção correta à instrução correta nos computadores em pipeline levou alguns projetistas de computador a relaxarem esse requisito em casos não críticos → alguns processadores são considerados como tendo Interrupções Imprecisas ou Exceções Imprecisas

Nesses processadores, é possível apenas parar o pipeline, salvar o estado e a(s) causa(s) da(s) exceção(ões); daí é deixado para o tratador de exceções verificar quais instruções causaram exceções e quais instruções devem ser completadas ou descartadas

Simplifica o hardware, mas exige software de tratamento mais complexo e é inviável em pipelines com execução múltipla fora de ordem

6 Estendendo o pipeline para lidar com operações de Ponto Flutuante

⇒ É pouco prático exigir que todas as operações de ponto flutuante MIPS concluam em um ciclo de clock ou ainda em dois → teríamos que aceitar um clock lento ou usar enorme quantidade de lógica nas unidades de ponto flutuante ou ambos

⇒ Em vez disso, o pipeline de ponto flutuante permitirá uma latência maior para as operações

⇒ Mudanças importantes em relação ao pipeline de inteiros:

1) O ciclo EX pode ser repetido tantas vezes quantas forem necessárias para concluir a operação

2) Pode haver múltiplas unidades funcionais de ponto flutuante

⇒ Haverá um stall se a instrução a ser despachada causar um hazard estrutural para a unidade funcional que ela utiliza ou se ela causar um hazard de dados

⇒ Vamos considerar que existam quatro unidades funcionais separadas em nossa implementação MIPS:

1. A unidade de inteiros principal que trata de loads e stores, operações de ALU com inteiros e desvios

2. Multiplicador de ponto flutuante e inteiros

3. Somador de ponto flutuante que trata de adição, subtração e conversão de PF

4. Divisor de ponto flutuante e inteiros

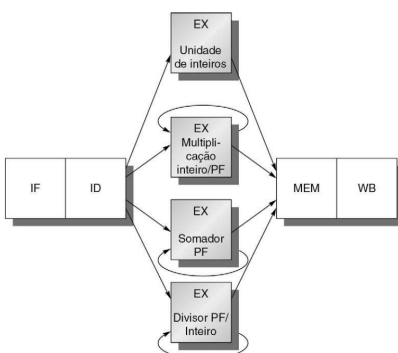


Figura 32: O pipeline MIPS com três unidades funcionais de ponto flutuante adicionais, em não pipeline. Como somente uma instrução é despachada em cada ciclo de clock, todas as instruções passam pelo pipeline-padrão para as operações com inteiros. As operações de ponto flutuante simplesmente fazem o loop quando atingem o estágio EX. Depois de terminarem o estágio EX, elas prosseguem para MEM e WB, para concluir a execução

⇒ Na representação acima, como estamos considerando que EX não está em pipeline, nenhuma outra instrução usando uma unidade funcional poderá ser despachada até que a instrução anterior usando essa mesma unidade funcional saia do EX → conflito estrutural entre instruções subsequentes que usam a mesma unidade funcional

⇒ Podemos generalizar a estrutura do pipeline de PF mostrada na figura anterior para permitir o pipelining de alguns estágios e múltiplas operações em andamento

⇒ Para descrever tal pipeline, temos que definir tanto a latência das unidades funcionais e o intervalo de iniciação ou intervalo de repetição

⇒ **Latência**: número de ciclos intercalados entre uma instrução que produz um resultado e uma instrução que usa o resultado

⇒ **Intervalo de iniciação ou de repetição**: número de ciclos que precisam passar entre o despacho de duas operações de determinado tipo

⇒ Exemplo da estrutura de pipeline que será usada:

Unidade funcional	Latência	Intervalo de iniciação
ALU de inteiros	0	1
Memória de dados (loads de inteiros e PF)	1	1
Adição de PF	3	1
Multiplicação de PF (também multiplicação de inteiros)	6	1
Divisão de PF (também divisão de inteiros)	24	25

Figura 33: Permite até 4 adições de PF pendentes, 7 multiplicações de PF/inteiros pendentes e 1 divisão de PF

⇒ Com essa definição de latência, as operações da ALU com inteiros possuem uma latência 0, pois os resultados podem ser usados no próximo ciclo de clock, e os loads têm uma latência de 1, pois seus resultados podem ser usados após um ciclo intermediário

⇒ Como a maioria das operações consome seus operandos no início de EX, a latência normalmente é o número de estágios após o EX em que uma instrução produz um resultado — por exemplo, zero estágios para operações da ALU e um estágio para loads

⇒ A principal exceção são os stores, que consomem o valor sendo armazenado um ciclo depois. Daí a latência para um store, para o valor sendo armazenado, mas não para o registrador de endereço básico, é de um ciclo a menos

⇒ A latência do pipeline é basicamente igual a um ciclo a menos que a profundidade do pipeline de execução, que é o número de estágios do estágio EX até o estágio que produz o resultado

⇒ A penalidade para a taxa de clock mais rápida é, portanto, a latência maior para as operações

⇒ Para a estrutura na próxima figura, vamos precisar da introdução de registradores de pipeline adicionais (ex: A1/A2, A2/A3, A3/A4) e a modificação das conexões com esses registradores

⇒ ID/EX precisa ser expandido para conectar o ID a: EX, A1, M1 e DIV

⇒ Como somente uma operação pode estar em um estágio de pipe de cada vez, a informação de controle pode ser associada ao registrador no início do estágio

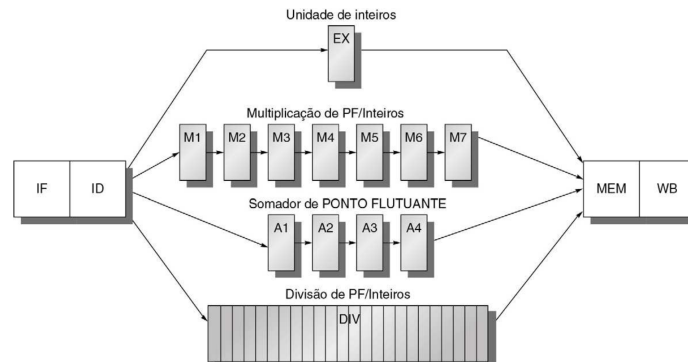


Figura 34: O multiplicador e o somador de PF estão totalmente em pipeline e possuem uma profundidade de 7 e 4 estágios, respectivamente. O divisor de PF não está em pipeline, mas exige 24 ciclos de clock para concluir. Para operações da ALU com inteiros, a profundidade do pipeline de execução é sempre 1 e a próxima instrução pode usar os resultados

⇒ No esquema acima, latência nas instruções entre o despacho de uma operação de PF e o uso do resultado dessa operação sem incorrer em um hazard RAW é determinada pelo número de ciclos gastos nos estágios de execução → ex: a quarta instrução após uma adição de PF pode usar o resultado da adição de PF (uma adição de PF leva 4 ciclos para ser completada)

MUL.D	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADD.D		IF	ID	A1	A2	A3	A4	MEM	WB		
L.D			IF	ID	EX	MEM	WB				
S.D				IF	ID	EX	MEM	WB			

Figura 35: Exemplo operações de PF independentes

⇒ A latência maior das operações de PF aumenta a frequência dos hazards RAW e stalls resultantes

6.1 Hazards e adiamentos nos pipelines de latência maior

1. Como a unidade de divisão não está em pipeline, podem ocorrer hazards estruturais. Estes precisarão ser detectados, e o despacho das instruções terá de ser atrasado

2. Como as instruções possuem tempos de execução variáveis, o número de escritas necessárias nos registradores em um ciclo pode ser maior do que 1

3. Hazards WAW são possíveis, pois as instruções não atingem mais o WB em ordem. Observe que os hazards WAR não são possíveis, pois as leituras de registradores sempre ocorrem em ID

4. As instruções podem ser completadas em uma ordem diferente das que foram despachadas, causando problemas com exceções
5. Devido à latência maior das operações, os stalls para os hazards RAW serão mais frequentes

Instrução	Número do ciclo de clock																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L.D F4,0(R2)	IF	ID	EX	MEM	WB												
MUL.D F0,F4,F6		IF	ID	stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
ADD.D F2,F0,F8			IF	stall	ID	stall	stall	stall	stall	stall	stall	A1	A2	A3	A4	MEM	WB
S.D F2,0(R2)					IF	stall	stall	stall	stall	stall	stall	ID	EX	stall	stall	stall	MEM

Figura 36: Exemplo com stalls que surgem de hazards RAW

⇒ Acima, cada instrução nessa sequência é dependente da anterior e prossegue assim que os dados estão disponíveis, o que assume que o pipeline possui forwarding completo; além disso, o S.D precisa ser adiado por um ciclo extra, de modo que seu MEM não entre em conflito com o ADD.D

Instrução	Número do ciclo de clock										
	1	2	3	4	5	6	7	8	9	10	11
MUL.D F0,F4,F6	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
ADD.D F2,F4,F6				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
L.D F2,0(R2)							IF	ID	EX	MEM	WB

Figura 37: Três instruções desejam realizar um write-back no Banco de Registradores de PF simultaneamente (ciclo 11): Hazard Estrutural!

⇒ Acima, Embora MUL.D, ADD.D e L.D estejam no estágio MEM no ciclo de clock 10, somente o L.D realmente utiliza a memória, de modo que não existe um hazard estrutural para MEM

⇒ Para ilustrar hazards WAW considere o exemplo da figura acima:

Se a instrução L.D fosse despachada um ciclo antes e tivesse um destino de F2, ela criaria um hazard WAW, pois escreveria no registrador F2 um ciclo antes do ADD.D → isso é incorreto, pois pela ordem do programa o dado que deve ficar em F2 após essas duas instruções serem concluídas é o dado escrito pelo L.D e não o dado escrito pelo ADD.D! → por isso devemos detectar hazards WAW e, no caso do exemplo, nos certificar de que o resultado do L.D aparece em F2 quando terminarmos essa sequência

6.2 Hazards de dados: RAW, WAW e WAR

⇒ Em nossa implementação do pipeline de inteiros é possível haver: RAW

⇒ Em nossa implementação do pipeline de PF/inteiros, vista na seção anterior, é possível haver: RAW e WAW

⇒ Considere duas instruções i e j, com i precedendo j na ordem do programa

RAW (Read After Write — leitura após escrita): j tenta ler um fonte antes que i escreva nele, de modo que j apanha incorretamente o valor antigo

WAW (Write After Write — escrita após escrita): j tenta escrever um operando antes que ele seja escrito por i. As escritas acabam sendo realizadas na ordem errada, deixando o valor escrito por i em vez do valor escrito por j no destino

WAR (Write After Read — escrita após leitura): j tenta escrever um destino antes que seja lido por i, de modo que i incorretamente apanha o valor novo → **não** ocorrem em nossas implementações de pipeline (PF e inteiros) pois todas as leituras de registradores vêm cedo (em ID) e todas as escritas em registradores vêm tarde (em WB)

Referências

FERREIRA, Mateus Felipe Tymburibá. AOC1. 2023. Slides de aula.

PATTERSON, David A.; HENNESSY, John L. ORGANIZAÇÃO E PROJETO DE COMPUTADORES: A INTERFACE HARDWARE/SOFTWARE. 4. ed. [S. l.]: CAMPUS, 2014.

HENNESSY, John L.; PATTERSON, David A. Arquitetura de Computadores: Uma Abordagem Quantitativa. 5. ed. [S. l.]: Elsevier Editora Ltda., 2014.